Battery LCA Tool

End-to-End Data Flow Walkthrough

From Login to Brightway Export — All Tables, All Scenarios

February 25, 2026

Scope of this document

This document traces a **single concrete scenario** — manufacturer *Foulan Foulani* logging in, working within a project created for him by the admin, creating the *Active Cathode Material* process from the Foulan 2025 SI2 dataset, validating it, mapping it to Brightway, and exporting it — and shows **exactly which rows are written to which tables at each step**, including every table in the schema: `app_user`, `project`, `project_member`, `catalog_set`, `flow_catalog`, `flow_allowed_unit`, `unit_catalog`, `data_origin_catalog`, `region_catalog`, `process_template`, `template_expected_exchange`, `battery_model`, `battery_model_revision`, `model_parameter`, `process_instance`, `process_exchange`, `process_link`, `validation_rule`, `validation_run`, `validation_issue`, `artifact`, `import_job`, `export_job`, `bw_mapping_candidate`, `bw_mapping_selection`.
Where a scenario can go two ways (e.g. validation pass vs. fail, mapping found vs. not found), **both paths are shown**.

Key design principle: Projects are admin-owned

Projects are always created by the admin, never by manufacturers. The admin creates a project (typically representing a study scope or client engagement), then assigns one or more manufacturer users to it via `project_member`. Manufacturers log in and find their assigned projects waiting for them. They then create battery models, revisions, and process data *within* those projects. This separation ensures that the admin controls project naming, scoping, and access — manufacturers only contribute data.

Contents

1 System Bootstrap — Tables Set Up Before Any User Logs In **5**
- 1.1 catalog_set — The Flow Namespace . . . 5
- 1.2 artifact_type Enum — Permitted Values . . . 6
- 1.3 unit_catalog — Permitted Units with SI Conversion Factors . . . 7
- 1.4 region_catalog — Geographic Codes . . . 7
- 1.5 data_origin_catalog — Provenance Categories . . . 8
- 1.6 flow_catalog — The Master Flow Vocabulary . . . 8
- 1.7 flow_allowed_unit — Unit Restrictions per Flow . . . 11
- 1.8 validation_rule — Machine-Readable Constraints . . . 12
- 1.9 process_template and template_expected_exchange . . . 13

2 Step 1 — Admin Creates a Manufacturer Account **17**

3 Step 2 — Admin Creates a Project and Assigns Foulan 18

4 Step 3 — Foulan Logs In and Creates a Battery Model 21
4.1 battery_model . . . 21
4.2 battery_model_revision . . . 22

5 Step 4 — Data Entry: Two Paths to the Same Tables 22
5.1 Path A — Manual Entry . . . 23
5.1.1 model_parameter — Set by Foulan in the Parameters Panel . . . 23
5.1.2 process_instance — Foulan Creates a Process . . . 24
5.1.3 process_exchange — Foulan Enters Exchanges Row by Row . . . 24
5.2 Path B — File Import . . . 27
5.2.1 artifact — The File Record . . . 27
5.2.2 import_job — Tracking the Parse Operation . . . 28
5.2.3 model_parameter — Created by the Parser from the File Header . . . 29
5.2.4 process_instance and process_exchange in Path B . . . 29
5.2.5 Scenario A — Normal Technosphere Input . . . 30
5.2.6 Scenario B — Negative Amount from Ecoinvent (Preserved As-Is) . . . 31
5.2.7 Scenario C — Biosphere (Elementary) Exchange . . . 31
5.2.8 Scenario D — Production (Reference) Flow . . . 32
5.3 process_link — Explicit Graph Edges (Optional) . . . 33

6 Step 5 — Validation Run 34
6.1 validation_run — The Snapshot Record . . . 34
6.2 validation_issue — One Row Per Finding . . . 34
6.2.1 Scenario A — Validation Passes (Informational Items Only) . . . 35
6.2.2 Scenario B — Validation Fails (Error Found) . . . 35
6.2.3 Scenario C — Missing Reference Flow . . . 36

7 Step 6 — Brightway Mapping 37
7.1 mapping_job — Async Candidate Generation Tracker . . . 37
7.2 bw_mapping_candidate — Automated Suggestions . . . 38
7.3 bw_mapping_selection — Foulan's Confirmed Choices . . . 39
7.3.1 Scenario A — Foulan confirms a background mapping . . . 39
7.3.2 Scenario B — Foreground flow (no background lookup) . . . 39
7.3.3 Scenario C — Unmappable flow . . . 40

8 Step 7 — Export 41
8.1 export_job — Tracking the Export Operation . . . 41
8.1.1 Scenario A — Export Succeeds . . . 41
8.1.2 Scenario B — Export Fails (Pending Mappings) . . . 42
8.2 Freezing the Revision After Export . . . 42
8.3 Admin Unfreeze — Recovery Path . . . 43

9 Step 8 — Monte Carlo Parametric Export 44

10 Complete Table-to-Step Cross-Reference 45

11 Schema Changes from Corpus Scan Analysis (Sessions 4–6) 46
11.1 Q11 — Output Type on process_exchange . . . 46
11.2 Q12 — Sign Convention and amount_is_ecoinvent_signed . . . 47
11.3 Q13 — ActivityNames XML Parser Fix (Pending) . . . 47
11.4 Q14 — Formula Handling: Use Pre-Computed Amount . . . 48

11.5 Corpus-Driven Seed Data Changes . . . 48

12 Brightway Export Layer — Implementation Note 48
12.1 What needs to be specified . . . 48
12.2 Recommended approach . . . 49

13 Key Design Clarifications 50
13.1 Admin vs. Manufacturer Responsibilities . . . 50
13.2 Why artifact and import_job / export_job are separate tables . . . 50
13.3 Why bw_mapping_candidate and bw_mapping_selection are separate . . . 51
13.4 What mapping_status values mean at export time . . . 51
13.5 How validation_issue links to specific data . . . 51

14 Technology Stack 52
14.1 System Architecture . . . 52
14.2 Frontend Technologies . . . 52
14.3 Backend — Supabase . . . 53
14.4 Processing Service — FastAPI on Railway . . . 54
14.5 Key Architectural Decisions . . . 55
14.6 Infrastructure Cost . . . 56
14.7 Recommended Build Order . . . 57
14.8 Development Tooling . . . 58

15 Admin Panel — Screens, Actions, and Data Access 59
15.1 Screen 1 — User Management . . . 59
15.1.1 User list view . . . 59
15.1.2 Invite a new user . . . 60
15.1.3 Change a user's role . . . 60
15.1.4 Deactivate a user . . . 60
15.2 Screen 2 — Project Management . . . 61
15.2.1 Project list view . . . 61
15.2.2 Create a new project . . . 61
15.2.3 Manage project membership . . . 62
15.2.4 Archive a project . . . 62
15.3 Screen 3 — Catalog Management . . . 62
15.3.1 Flow Catalog tab . . . 63
15.3.2 Unit Catalog tab . . . 64
15.3.3 Region Catalog tab . . . 64
15.3.4 Data Origin Catalog tab . . . 64
15.3.5 Validation Rules tab . . . 65
15.3.6 Process Templates tab . . . 65
15.4 Screen 4 — Global Data View . . . 66
15.4.1 All Projects Summary . . . 66
15.4.2 Cross-Project Flow Deduplication Review . . . 67
15.4.3 Revision Status Dashboard . . . 67
15.4.4 Mapping Coverage Report . . . 68
15.5 Screen 5 — Data Export . . . 68
15.5.1 Export a single revision (Brightway format) . . . 69
15.5.2 Export full project data as JSON . . . 69
15.5.3 Export catalog tables as CSV . . . 70
15.5.4 Import catalog updates from CSV . . . 70
15.6 Admin Panel — RLS and Edge Function Summary . . . 70

15.7 Build Order Addition — Admin Panel Phases 71

1 System Bootstrap — Tables Set Up Before Any User Logs In

Before any manufacturer can use the tool, an administrator runs the initial setup. This populates the *configuration layer* tables that all projects share. These are one-time operations that do not change with individual user activity.

Important: not all of these tables need to be fully pre-populated

A common misconception when reading this section is that all of these tables must contain complete data before any manufacturer can log in. That is only true for three of them. The table below clarifies which tables are genuinely required at bootstrap, and which grow dynamically as manufacturers use the tool:

Table	Pre-loaded?	Reason
`unit_catalog`	**Yes, fully**	Pre-seeded from ecoinvent corpus: 21 exchange units confirmed across 26,533 datasets. Never changes post-deployment.
`region_catalog`	**Yes, fully**	Pre-seeded from all 541 geographies in ecoinvent Geographies.xml (not just the 335 appearing in the corpus). Never changes post-deployment.
`catalog_set`	**Yes, 1 row**	Must exist before any flow can be created.
`data_origin_catalog`	Partially	A few default categories shipped; more added as needed.
`validation_rule`	Partially	Core rules shipped; new rules added as tool evolves.
`flow_catalog`	**Starts empty**	Grows automatically as manufacturers import or enter data.
`flow_allowed_unit`	**Starts empty**	Created alongside each new flow.
`process_template`	**Starts empty**	Created reactively as common patterns emerge.
`template_expected_exchange`	**Starts empty**	Created alongside each new template.

1.1 catalog_set — The Flow Namespace

What it is

`catalog_set` defines a named, versioned collection of flows and rules. Think of it as the *vocabulary* the tool uses. All projects in this deployment reference the `DEFAULT` catalog which ships pre-populated. A future deployment for a different background database (e.g. ecoinvent 3.10) would create a second row here.

Design note: who creates a catalog_set and when?

A `catalog_set` is always created by an **admin**, never by a manufacturer. The `created_by FK → app_user` column is present and an RLS policy should restrict `INSERT` on this table to `role = admin`. Manufacturers get `SELECT` only.
Three realistic deployment scenarios exist:

- **One shared catalog for all projects (recommended default):** Admin creates a

single DEFAULT row at deployment. All manufacturers and all projects share it. New flows auto-created by any manufacturer's import are visible to all, which means BW mappings done once benefit everyone.

- **One catalog per project (stronger isolation):** Admin creates a new catalog_set row each time a project is created. Projects are fully isolated — flows created in one project do not appear in another. This trades the shared-vocabulary benefit for confidentiality.
- **Versioned catalogs (for reproducibility):** Admin creates a new catalog_set (e.g. DEFAULT_v2) before making breaking changes to flow definitions. Frozen revisions continue referencing the old catalog version indefinitely.

```sql
INSERT INTO catalog_set (catalog_set_id, name, description, created_by,
    created_at)
VALUES (
  'cat-0001-uuid',
  'DEFAULT',
  'Foulan 2025 SI2 flow namespace. Ships with installation.',
  'admin-user-uuid',
  NOW()
);
```

catalog_set_id	name	description
cat-0001-uuid	DEFAULT	Foulan 2025 SI2 flow namespace

1.2 artifact_type Enum — Permitted Values

Full set of permitted artifact_type values

The artifact table uses an ENUM column artifact_type. The complete set of permitted values, confirmed as part of the pre-build decisions, is:

Value	Meaning
import	An xlsx or other source file uploaded by a manufacturer for parsing
export	A Brightway xlsx file generated by the export engine for a manufacturer
parameter_set	A Monte Carlo JSON results file from the /montecarlo endpoint
admin_export	A full project JSON snapshot generated by the admin export screen

Every artifact row has a revision_id (NOT NULL FK to battery_model_revision). For admin_export artifacts that cover a whole project, one artifact row is created **per revision** exported, each with its own revision_id. There is no case where revision_id is NULL.

1.3 unit_catalog — Permitted Units with SI Conversion Factors

What it is

`unit_catalog` is a static lookup of every unit the tool accepts, its physical dimension, and its conversion factor to SI base. It is used at *export time* to convert `quantity_user` (stored as-entered) into canonical SI amounts for Brightway. Pre-seeded at installation from the 21 exchange units confirmed across all 26,533 ecoinvent datasets in the corpus scan; users never write to it directly.

unit_id	symbol	dimension	factor_to_si	description
u-001	kg	mass	1.0	Kilogram (SI base)
u-002	unit	count	1.0	Dimensionless count
u-003	kWh	energy	3 600 000.0	Kilowatt-hour → Joules
u-004	m3	volume	1.0	Cubic metre (SI base)
u-005	MJ	energy	1 000 000.0	Megajoule → Joules
u-006	metric ton*km	transport	1 000.0	Tonne-kilometre → kg·m
u-007	m2	area	1.0	Square metre (SI base)
u-008	m2*year	area·time	1.0	Land occupation
u-009	ha	area	10 000.0	Hectare → m^2
u-010	km	length	1 000.0	Kilometre → metres
u-011	m	length	1.0	Metre (SI base)
u-012	kBq	radioactivity	1 000.0	Kilobecquerel → Bq
u-013	hour	time	3 600.0	Hour → seconds
u-014	m*year	length·time	1.0	Infrastructure occupation
u-015	m3*year	volume·time	1.0	Water occupation
u-016	Sm3	volume	1.0	Standard cubic metre (gas)
u-017	l	volume	0.001	Litre → m^3
u-018	kg*day	mass·time	86 400.0	Ecotoxicity exposure unit
u-019	person*km	transport	1.0	Passenger-kilometre
u-020	km*year	length·time	1.0	Infrastructure occupation
u-021	guest night	count	1.0	Accommodation unit

21 rows — complete set confirmed from ecoinvent corpus scan of 26,533 datasets.

1.4 region_catalog — Geographic Codes

What it is

`region_catalog` lists every geographic identifier the tool recognises. It is pre-seeded from ecoinvent's `Geographies.xml` master data file, which defines 541 geographies in total. This includes all ISO 3166-1 alpha-2 codes (`DE`, `CN`), ecoinvent custom shortcuts (`RER`, `GLO`, `RoW`), sub-national grids (`CN-CSG`, `US-WECC`), and IAI area aggregates. Seeding from the full 541-entry master file — rather than only the 335 geographies that appear in the current corpus — ensures that future ecoinvent releases or manufacturer processes referencing rare geographies do not fail foreign key checks.

code	name	is_ecoinvent_shortcut
DE	Germany	FALSE
GLO	Global	TRUE
RER	Europe	TRUE
RoW	Rest of World	TRUE
CN	China	FALSE
CN-CSG	China Southern Grid	TRUE
US-WECC	Western Electricity Coordinating Council	TRUE
RNA	North America (ecoinvent)	TRUE
IAI Area, Asia, without China and GCC	IAI Asia region	TRUE

541 rows total — seeded from ecoinvent Geographies.xml. 9 representative rows shown.

1.5 data_origin_catalog — Provenance Categories

What it is

Each exchange can carry a `data_origin_id` that says *how* the number was obtained. This table defines the permitted categories. It is configuration-driven so new categories (e.g. `simulation`) can be added without a schema change.

code	label	description
measured	Measured	Directly measured at plant
datasheet	Datasheet	Taken from supplier/product datasheet
literature	Literature	Sourced from peer-reviewed publication
estimate	Estimate	Engineering estimate; lower data quality
default	Default	Tool default value; not verified

1.6 flow_catalog — The Master Flow Vocabulary

What it is

`flow_catalog` is the central reference. Every material, energy carrier, emission, waste, water flow, and service flow the tool knows about lives here. When a user types *electricity* in an exchange row, it resolves to a `flow_id` from this table. The `UNIQUE(catalog_set_id, canonical_name, kind)` constraint (FIX-02) ensures there is exactly one row per named flow per catalog. `flow_id` is referenced as a FK by `process_exchange`, `flow_allowed_unit`, `template_expected_exchange`, `bw_mapping_candidate`, and `bw_mapping_selection` — making it one of the most widely referenced tables in the schema.

Design note: flow_catalog does NOT need to be pre-populated

It is **not standard LCA practice** to define all flows in advance. Tools like Brightway and SimaPro do not require a predefined flow list — flows are named freely during data entry and matched to background datasets during the mapping step. This tool follows the same model.
`flow_catalog` **starts empty** at deployment and grows in two ways:

- **On import:** when the parser encounters a flow name not already in the catalog, it calls the `create-flow` Edge Function (which runs with the service-role key) to insert a new `flow_catalog` row. The `dimension` is inferred from the unit string (e.g. `kg` → mass; `kWh` → energy; `m3` → volume). Every auto-creation is logged for admin review. The manufacturer never consciously creates catalog rows.
- **On manual entry:** when a manufacturer types a flow name in the exchange grid that is not found in the dropdown, the UI prompts them to confirm the dimension and unit, then calls the same `create-flow` Edge Function.

Why the Edge Function pattern? `flow_catalog` INSERT is restricted to `role = admin` in the RLS policy. Manufacturers run as `role = editor` and cannot INSERT directly. The `create-flow` Edge Function uses the service-role key and validates that the caller has an active session and the correct `catalog_set_id` before inserting. This keeps the RLS boundary intact while allowing the auto-creation behaviour that lazy importers require.
The **admin's ongoing task** is deduplication review — merging flows like `NiSO4` and `nickel_sulfate_solution` that different manufacturers created independently for the same physical substance. The `UNIQUE` constraint prevents exact duplicates automatically; near-duplicates require human judgment.
Unit safety (`flow_allowed_unit`) and dimension checking work fully once a flow has a confirmed `dimension`. For auto-created flows where the dimension could not be inferred, it is left NULL and the validation engine raises a `warning` until confirmed.
The rows shown in the SQL and tables below represent what the catalog *looks like after several imports* — not what ships at deployment.

Field definitions with keys.

Field	Type / role	Description
`flow_id`	UUID **(PK)**	Unique identifier. Referenced as FK by 5 other tables.
`catalog_set_id`	UUID **(FK → catalog_set)**	Which catalog namespace this flow belongs to.
`canonical_name`	TEXT	Lowercase underscore identifier, e.g. `cobalt_sulfate_primary`.
`display_name`	TEXT (nullable)	Human-readable label shown in UI dropdowns.
`kind`	ENUM	`material` / `energy` / `emission` / `waste` / `water` / `service` [v2 FIX-05]
`dimension`	ENUM	`mass` / `energy` / `volume` / `area` / `length` / `count`
`default_unit`	TEXT	Canonical unit for this flow, e.g. `kg`, `kWh`, `m3`.

is_elementary_flow	BOOLEAN DEFAULT FALSE	TRUE = biosphere (emission/resource); FALSE = technosphere or service.
cas_number	TEXT (nullable)	Optional CAS number for chemical identity.
created_at	TIMESTAMPTZ	Row creation timestamp.
UNIQUE(catalog_set_id, canonical_name, kind) [**FIX-02**]		

```sql
INSERT INTO flow_catalog
  (flow_id, catalog_set_id, canonical_name, display_name,
   kind, dimension, default_unit, is_elementary_flow, cas_number)
VALUES
  ('flw-niso4-uuid',  'cat-0001-uuid', 'nickel_sulfate_primary',
   'Nickel Sulfate (primary)', 'material', 'mass', 'kg', FALSE,
       '7786-81-4'),

  ('flw-coso4p-uuid', 'cat-0001-uuid', 'cobalt_sulfate_primary',
   'Cobalt Sulfate (primary)',  'material', 'mass', 'kg', FALSE,
       '10124-43-3'),

  ('flw-coso4r-uuid', 'cat-0001-uuid', 'cobalt_sulfate_recycled',
   'Cobalt Sulfate (recycled)', 'material', 'mass', 'kg', FALSE,
       '10124-43-3'),
  -- ^ same CAS as primary; disambiguated by canonical_name + kind [FIX-02]

  ('flw-elec-uuid',   'cat-0001-uuid', 'electricity',
   'Electricity (foreground)', 'energy',   'energy', 'kWh', FALSE, NULL),

  ('flw-ww-uuid',     'cat-0001-uuid', 'market_wastewater_avg',
   'Market for wastewater, average', 'water', 'volume', 'm3', FALSE, NULL),

  ('flw-cod-uuid',    'cat-0001-uuid', 'COD_chemical_oxygen_demand',
   'COD, Chemical Oxygen Demand', 'emission', 'mass', 'kg', TRUE, NULL),
  -- ^ is_elementary_flow = TRUE: this is a biosphere flow

  ('flw-mwf-uuid',    'cat-0001-uuid', 'mkt_metal_working_factory',
   'Market for metal working factory', 'service', 'count', 'unit', FALSE,
       NULL);
  -- ^ kind = 'service': added by FIX-05
```

Populated rows with all key columns shown.

flow_id (PK)	catalog_set_id (FK)	canonical_name	kind	dimension	unit	elem?
flw-niso4-uuid	cat-0001-uuid	nickel_sulfate_primary	material	mass	kg	F
flw-coso4p-uuid	cat-0001-uuid	cobalt_sulfate_primary	material	mass	kg	F
flw-coso4r-uuid	cat-0001-uuid	cobalt_sulfate_recycled	material	mass	kg	F
flw-elec-uuid	cat-0001-uuid	electricity	energy	energy	kWh	F

flw-ww-uuid	cat-0001-uuid	market_wastewater_avg	water	volume	m3	F
flw-cod-uuid	cat-0001-uuid	COD_chem_oxygen_demand	emission	mass	kg	**T**
flw-mwf-uuid	cat-0001-uuid	mkt_metal_working_factory	**service**	count	unit	F

493 flows in DEFAULT catalog. 7 representative rows shown.

How flow_id is used downstream

Once a flow is in `flow_catalog`, its `flow_id` is referenced in 5 places:

- `flow_allowed_unit.flow_id` (FK) — defines which units this flow accepts
- `template_expected_exchange.flow_id` (FK) — marks this flow as expected in a template
- `process_exchange.flow_id` (FK, nullable) — the exchange resolves to this flow
- `bw_mapping_candidate.flow_id` (FK) — automated BW match suggestions for this flow
- `bw_mapping_selection.flow_id` (FK) — Foulan's confirmed BW mapping for this flow

Deleting a flow from `flow_catalog` would cascade-fail on all five FK constraints, which is by design: flows are never deleted, only deprecated via a new catalog version.

1.7 flow_allowed_unit — Unit Restrictions per Flow

What it is

For each flow in `flow_catalog`, `flow_allowed_unit` lists the units a user is permitted to enter. The UI's unit dropdown is populated from this table. A user cannot enter `kWh` for a mass flow because no row with `unit = kWh` exists for `nickel_sulfate_primary`. This prevents dimensional errors before they reach the database.

Design note: created alongside each new flow (not pre-loaded)

`flow_allowed_unit` rows are created **at the same time** as their parent `flow_catalog` row, either by the import parser or by the UI when a manufacturer confirms a new flow's dimension. The parser derives the allowed units from the `dimension` value — all units in `unit_catalog` that share the same dimension are inserted as allowed. For example, a new flow with `dimension = mass` automatically gets `kg`, `g`, and `t` as allowed units.
This table therefore **starts empty** and is never manually pre-populated by the admin. Its contents are always a direct consequence of what is in `flow_catalog` and `unit_catalog`.

flow_id (canonical_name)	allowed units
nickel_sulfate_primary	kg, g, t
electricity	kWh, MJ, GJ
market_wastewater_avg	m3, L
COD_chemical_oxygen_demand	kg, g, mg

One row per (flow, unit) pair.

1.8 validation_rule — Machine-Readable Constraints

What it is

`validation_rule` stores the rules that the validation engine enforces. Each rule has a `code` (referenced in `validation_issue.code`), a `severity` (`error` blocks export; `warning` does not), and a `rule_json` blob that the validation engine reads to know what to check. Rules are config-driven so they can evolve without schema changes.

code	severity	description
MISSING_REF_FLOW	error	Every `process_instance` must have exactly one `process_exchange` with `output_type = reference`.
UNMAPPED_FLOW	warning	All `flow_id` values in `process_exchange` must have a `bw_mapping_selection` row that is not `pending`.
UNIT_MISMATCH	error	`user_unit` must be present in `flow_allowed_unit` for the associated `flow_id`.
NEGATIVE_FOREGROUND_QTY	error	`quantity_user` must be ≥ 0 for all exchanges where `amount_is_ecoinvent_signed = FALSE`. Foreground (manufacturer-entered) flows must always be positive; direction is carried by `exchange_direction`. Ecoinvent-imported rows are exempt.
NEGATIVE_REF_PRODUCT	error	No exchange with `output_type = reference` may have a negative `quantity_user`, regardless of source. A process producing a negative quantity of its own reference product is always a data error.
PARAM_RANGE	warning	`min_value` $\leq$ `value` $\leq$ `max_value` must hold for all `model_parameter` rows.
FORMULA_PRESERVED	info	Log when `formula_user` was captured from a non-trivial formula string. The formula is stored for audit only and is never re-evaluated at import.
FOREGROUND_CONFIRMED	info	Log flows with `mapping_status = foreground`; confirms that no background ecoinvent lookup was attempted for these flows.
ECOINVENT_NEGATIVE_AMT	info	Log ecoinvent-imported exchanges where `quantity_user < 0` (`amount_is_ecoinvent_signed = TRUE`). Informational only; these amounts are intentional and must not be altered.

Removed rules: NEGATIVE_QTY and FLIP_APPLIED

Two rules from earlier schema versions have been removed after the full ecoinvent corpus scan revealed they were based on incorrect assumptions.
NEGATIVE_QTY (formerly: error if any `quantity_user < 0`) has been replaced by two precise rules: `NEGATIVE_FOREGROUND_QTY` (applies only to manufacturer-entered foreground rows) and `NEGATIVE_REF_PRODUCT` (applies to all reference product outputs regardless of

source). The blanket rule was wrong because 50.3% of ecoinvent datasets contain intentionally negative exchange amounts — waste treatment inputs, avoided burdens, and soil metal uptake are all expressed as negative values by ecoinvent convention (and by SimaPro's convention for the same flow types).
FLIP_APPLIED (formerly: FIX-01 sign-flip applied by the import parser) has been removed entirely. The flip rule was always incorrect for ecoinvent-sourced data. Preserving amounts exactly as they appear in the source `.spold` file is the correct behaviour; no sign transformation should be applied at import. The `amount_is_ecoinvent_signed` flag on `process_exchange` allows the BW export layer to apply the correct A-matrix sign convention at export time rather than corrupting stored data at import time.

1.9 process_template and template_expected_exchange

What these two tables are and how they link

`process_template` defines a named blueprint for a standard battery process. `template_expected_exchange` defines the individual exchange slots that blueprint expects — one row per expected flow. The two tables are linked by a **foreign key**: `template_expected_exchange.template_id` → `process_template.template_id`. Each `template_expected_exchange` row also carries a second FK: `flow_id` → `flow_catalog.flow_id`, so the expected flow is always a catalogued, named flow rather than a free-text string.
When Foulan creates a `process_instance` and selects the `cathode_material` template, the UI queries `template_expected_exchange WHERE template_id = 'tpl-cathode-uuid'` to pre-populate the exchange rows. Validation later re-runs the same query to verify that all rows with `is_required = TRUE` are present.

Design note: templates are optional and created reactively (not pre-loaded)

Process templates are a **convenience feature**, not a structural requirement. `process_instance.template_id` is nullable — a manufacturer can create any process without selecting a template at all and simply add exchanges manually.
Neither `process_template` nor `template_expected_exchange` need to contain any rows before manufacturers begin working. Both tables **start empty**. They are populated by an admin *reactively* as common process patterns emerge from real usage, following a lifecycle like this:

1. A manufacturer enters a cathode material process from scratch, adding all exchanges manually.
2. The admin notices this process type will appear in many studies and creates a template from it, inserting into `process_template` and `template_expected_exchange`.
3. Future manufacturers creating the same process type select the template and get the exchange rows pre-populated.

This is the same model used by SimaPro's process library — templates accumulate from usage rather than being defined upfront. The rows shown below represent what the tables *look like after the tool has been in use*, not what ships at deployment.
Note also that because `template_expected_exchange.flow_id` is a FK to `flow_catalog`, a template can only reference flows that already exist in the catalog. This means templates can only be created for process types whose flows have already appeared in at least one import or manual entry session.

process_template — primary key and foreign keys.

Field	Type / role	Description
template_id	UUID **(PK)**	Unique identifier for the template
catalog_set_id	UUID **(FK → catalog_set)**	Which catalog this template belongs to
stage	ENUM	manufacturing / use / end_of_life
canonical_name	TEXT	Stable internal identifier
ui_label	TEXT (nullable)	Display label shown in the UI
ui_helptext	TEXT (nullable)	Optional instructions for the user

```sql
INSERT INTO process_template
  (template_id, catalog_set_id, stage, canonical_name, ui_label,
      ui_helptext)
VALUES
  ('tpl-cathode-uuid', 'cat-0001-uuid', 'manufacturing',
   'cathode_material',
   'Active Cathode Material',
   'Synthesis of NMC or LFP cathode powder. Requires metal sulfate inputs, '
   'process electricity, and a wastewater output.'),

  ('tpl-cell-uuid', 'cat-0001-uuid', 'manufacturing',
   'cell_assembly',
   'Cell Assembly',
   'Dry-room assembly of electrodes, separator, and electrolyte into a
      cell.'),

  ('tpl-pack-uuid', 'cat-0001-uuid', 'manufacturing',
   'pack_assembly',
   'Pack Assembly',
   'Integration of cells into modules and modules into a battery pack.'),

  ('tpl-hydro-uuid', 'cat-0001-uuid', 'end_of_life',
   'hydromet_recycling',
   'Hydrometallurgical Recycling',
   'Recovery of Co, Ni, Li, Mn via hydrometallurgical route.');
```

template_id	stage	canonical_name	catalog_set_id (FK)
tpl-cathode-uuid	manufacturing	cathode_material	cat-0001-uuid
tpl-cell-uuid	manufacturing	cell_assembly	cat-0001-uuid
tpl-pack-uuid	manufacturing	pack_assembly	cat-0001-uuid
tpl-hydro-uuid	end_of_life	hydromet_recycling	cat-0001-uuid

template_expected_exchange — primary key and both foreign keys.

Field	Type / role	Description
expected_id	UUID **(PK)**	Unique identifier for this expected exchange slot

template_id	UUID **(FK → process_template)**	Which template this slot belongs to
flow_id	UUID **(FK → flow_catalog)**	Which catalogued flow is expected
direction	ENUM	input / output
is_required	BOOLEAN	TRUE = at least one occurrence must be entered
group_key	TEXT (nullable)	Groups repeatable entries (e.g. metal_inputs)
min_occurs	INT	Minimum times this group must appear
max_occurs	INT (nullable)	NULL = unbounded
display_order	INT (nullable)	UI row ordering
ui_label	TEXT (nullable)	Label override for this slot

```sql
-- Every row references template_id = 'tpl-cathode-uuid'  (FK ->
    process_template)
-- Every row references a flow_id from flow_catalog         (FK ->
    flow_catalog)
INSERT INTO template_expected_exchange
  (expected_id, template_id, flow_id,
   direction, is_required, group_key,
   min_occurs, max_occurs, display_order, ui_label)
VALUES
  -- Reference / production output
  ('tee-001', 'tpl-cathode-uuid',
   (SELECT flow_id FROM flow_catalog WHERE
       canonical_name='Active_Cathode_Material'),
   'output', TRUE, 'reference',
   1, 1, 1, 'Active Cathode Material (reference flow)'),

  -- Metal sulfate inputs (repeatable group: min 1 occurrence, unbounded)
  ('tee-002', 'tpl-cathode-uuid',
   (SELECT flow_id FROM flow_catalog WHERE
       canonical_name='nickel_sulfate_primary'),
   'input', TRUE, 'metal_inputs',
   1, NULL, 2, 'Nickel source'),

  ('tee-003', 'tpl-cathode-uuid',
   (SELECT flow_id FROM flow_catalog WHERE
       canonical_name='cobalt_sulfate_primary'),
   'input', TRUE, 'metal_inputs',
   1, NULL, 3, 'Cobalt source'),

  -- Electricity: required, exactly once
  ('tee-004', 'tpl-cathode-uuid',
   (SELECT flow_id FROM flow_catalog WHERE canonical_name='electricity'),
   'input', TRUE, 'utilities',
   1, 1, 4, 'Process electricity'),

  -- Wastewater: optional output
  ('tee-005', 'tpl-cathode-uuid',
   (SELECT flow_id FROM flow_catalog WHERE
       canonical_name='market_wastewater_avg'),
   'output', FALSE, 'waste_outputs',
   0, NULL, 5, 'Wastewater to treatment (if applicable)');
```

expected_	template_id (FK)	flow_id (FK)	dir	required?	group_key	min
tee-001	tpl-cathode-uuid	→ Active_Cathode_Material	output	**YES**	reference	1
tee-002	tpl-cathode-uuid	→ nickel_sulfate_primary	input	**YES**	metal_inputs	1
tee-003	tpl-cathode-uuid	→ cobalt_sulfate_primary	input	**YES**	metal_inputs	1
tee-004	tpl-cathode-uuid	→ electricity	input	**YES**	utilities	1
tee-005	tpl-cathode-uuid	→ market_wastewater_avg	output	no	waste_outputs	0

How the three-table chain is used at runtime

1. Foulan selects template `cathode_material` when creating a process.
2. Frontend queries: `SELECT tee.*, fc.canonical_name, fc.default_unit`
   `FROM template_expected_exchange tee`
   `JOIN flow_catalog fc ON fc.flow_id = tee.flow_id`
   `WHERE tee.template_id = 'tpl-cathode-uuid'`.
3. UI pre-populates 5 exchange rows using the result. Foulan fills in quantities.
4. `process_instance.template_id` is set to `'tpl-cathode-uuid'` (FK) so the link is permanent.
5. At validation, the engine re-runs step 2 and checks that for every row with `is_required = TRUE`, at least `min_occurs` matching `process_exchange` rows exist for the process.

2 Step 1 — Admin Creates a Manufacturer Account

STEP 1 — Admin invites Foulan Foulani via the user management screen

↓

What triggers this

The admin opens the **User Management** screen and enters `Foulan.Foulani@example.de` with role `editor`. This calls the `invite-user` Supabase Edge Function.

What the Edge Function does.

1. Verifies the calling user has `role = admin` in `app_user`.
2. Calls `supabase.auth.admin.inviteUserByEmail({email, data: {role: 'editor'}})`.
3. Supabase Auth sends a magic-link invite email to Foulan.
4. A webhook fires `on-user-created` when Foulan accepts the invite.

Rows written.

```sql
INSERT INTO app_user (user_id, email, display_name, role, created_at)
VALUES (
  'usr-Foulan-uuid',
  'Foulan.Foulani@example.de',
  'Foulan Foulani',
  'editor',
  '2024-01-10 09:00:00+00'
);
```

user_id	email	display_name	role	last_login_at
usr-Foulan-uuid	Foulan.Foulani@example.de	Foulan Foulani	editor	NULL (not logged in yet)

No password_hash stored

Authentication is delegated to Supabase Auth (OIDC). The `app_user` table never stores a password. It holds only the resolved identity and display metadata for audit trails and RLS policies.

3 Step 2 — Admin Creates a Project and Assigns Foulan

STEP 2 — Admin creates the NMC811 project and assigns Foulan as a member

↓

Projects are always created by the admin

Manufacturers cannot create projects. The admin is solely responsible for creating projects, setting their scope and name, and deciding which manufacturer users are assigned to them. This ensures project naming is consistent, access is intentional, and no manufacturer can unilaterally spin up new project workspaces.
The `project` table's RLS policy restricts `INSERT` to `role = admin`. Manufacturers have `SELECT` on projects they are members of, and `INSERT/UPDATE` on the data *within* those projects (models, revisions, processes), but never on the project record itself.

What happens. The admin opens the **Project Management** screen, fills in the project details for the NMC811 study, and selects Foulan Foulani as the assigned manufacturer. The frontend calls the `create-project` Edge Function (which runs with the service-role key to bypass the manufacturer-blocking RLS policy on `project`).

```sql
-- Called by the admin via the create-project Edge Function.
-- RLS on project restricts INSERT to role = admin.
INSERT INTO project
  (project_id, name, description, created_by, created_at, archived)
VALUES (
  'proj-0001-uuid',
  'NMC811 Parametric LCA -- Foulan 2025',
  'Parametric LCA model for NMC811 battery pack',
  'admin-user-uuid',    -- FK -> app_user (the admin, NOT Foulan)
  NOW(),
  FALSE
);
```

Schema change from earlier versions: created_by vs owner_id

In earlier versions of this schema the `project` table used `owner_id` pointing to the manufacturer. This has been corrected: `created_by` now points to the **admin** who created the project. Ownership in the sense of "who works on this project" is expressed entirely through `project_member` rows. There is no single *owner* manufacturer — there can be multiple assigned manufacturers, and the admin retains control regardless.

Admin assigns Foulan to the project. Immediately after creating the project the admin adds Foulan as a member. This is also done through the admin interface and Edge Function, not by Foulan.

INSERT: project_member — admin assigns Foulan with role "manufacturer"

```sql
-- The admin assigns Foulan to the project.
-- Foulan did not request this; he will simply see the project when he logs
    in.
INSERT INTO project_member
```

```
  (project_id, user_id, role, assigned_by, created_at)
VALUES (
  'proj-0001-uuid',
  'usr-Foulan-uuid',    -- FK -> app_user (Foulan)
  'manufacturer',       -- role within this project
  'admin-user-uuid',    -- FK -> app_user (the admin who made the assignment)
  NOW()
);
```

Schema change: project_member gains assigned_by column

`project_member` gains an `assigned_by` column (FK → `app_user`) recording which admin performed the assignment. This provides a full audit trail: who was given access to which project, by whom, and when. The column is NOT NULL and must be set at insert time by the Edge Function using the admin's `auth.uid()`.
The `role` values for `project_member` are also clarified:

- **manufacturer** — a manufacturer user who contributes data to this project
- **reviewer** — full read access to all data within the project (models, revisions, processes, exchanges, parameters, formulas, mapping selections, validation runs, and artifacts) but cannot trigger validation, export, or write anything. A single RLS rule — `project_member` membership grants `SELECT` on all project data — covers this without per-table carve-outs
- **admin** — an admin explicitly listed as a member for visibility (optional; admins can always access all projects via service-role regardless)

The old `owner` role is removed from `project_member`: ownership is expressed by the `project.created_by` column, not by a membership role.

What Foulan sees when he logs in. When Foulan later accepts the invite email and logs in (Step 2b), Supabase Auth issues a JWT with `auth.uid() = 'usr-Foulan-uuid'`. The RLS policy on `project` allows him to `SELECT` any project where a `project_member` row exists for his `user_id`. He sees the *NMC811 Parametric LCA — Foulan 2025* project on his dashboard, ready for him to start adding battery models.

project_id	name	created_by	archived
proj-0001-uuid	NMC811 Parametric LCA — Foulan 2025	admin-user-uuid	FALSE

project_id	user_id	role	assigned_by	created_at
proj-0001-uuid	usr-Foulan-uuid	manufacturer	admin-user-uuid	2024-01-10 09:10:00+00

How RLS uses project_member

Every query on `battery_model`, `process_instance`, `process_exchange` etc. includes a Postgres RLS policy:

```
-- Example RLS policy on battery_model:
USING (project_id IN (
```

```
  SELECT project_id FROM project_member WHERE user_id = auth.uid()
))
```

Foulan can only see rows belonging to projects where he has been assigned as a member by the admin. A second manufacturer logging in sees none of Foulan's data unless the admin has explicitly assigned them to the same project.
The RLS policy on `project` itself:

```
-- SELECT: manufacturers see their assigned projects only
-- INSERT/UPDATE/DELETE: restricted to admin role
CREATE POLICY project_select ON project FOR SELECT
  USING (
    project_id IN (
      SELECT project_id FROM project_member WHERE user_id = auth.uid()
    )
    OR (SELECT role FROM app_user WHERE user_id = auth.uid()) = 'admin'
  );

CREATE POLICY project_insert ON project FOR INSERT
  WITH CHECK (
    (SELECT role FROM app_user WHERE user_id = auth.uid()) = 'admin'
  );
```

4 Step 3 — Foulan Logs In and Creates a Battery Model

STEP 3 — Foulan logs in, finds his assigned project, creates a battery model and revision

↓

On login. Foulan accepts the invite email and sets his password. Supabase Auth issues a JWT. The `on-user-created` Edge Function updates `last_login_at`. The JWT contains `auth.uid() = 'usr-Foulan-uuid'`; every subsequent database query runs under this identity, and Row Level Security policies use this to restrict data access.

Foulan's view on login. He opens the tool and sees the project *NMC811 Parametric LCA — Foulan 2025* on his dashboard — created and assigned to him by the admin. He does **not** have a "New Project" button; only a "New Model" button within existing assigned projects.

4.1 battery_model

What it is

`battery_model` is the stable identity of a battery dataset. It is intentionally lightweight — most work happens in `battery_model_revision`. The model points to `project` and records the chemistry, functional unit, and creator. It never changes once created; revisions capture all evolution.

```sql
-- Foulan creates a new model inside the project the admin assigned to him.
-- The RLS policy on battery_model checks that proj-0001-uuid is in Foulan's
-- project_member rows before allowing this INSERT.
INSERT INTO battery_model
  (model_id, project_id, name, chemistry, functional_unit, created_by,
      created_at)
VALUES (
  'mdl-0001-uuid',
  'proj-0001-uuid',    -- FK -> project (admin-created project)
  'NMC811 Parametric Model',
  'NMC811',
  '1 kWh battery pack',
  'usr-Foulan-uuid',      -- FK -> app_user (Foulan is the model creator)
  NOW()
);
```

model_id (PK)	project_id (FK)	name	chemistry	functional_unit
mdl-0001-uuid	proj-0001-uuid	NMC811 Parametric Model	NMC811	1 kWh battery pack

4.2 battery_model_revision

What it is

`battery_model_revision` is the unit of work, validation, and export. Foulan starts with revision 1, status `draft`. The partial unique index on `(model_id) WHERE is_active = TRUE` ensures exactly one revision is active at a time. When Foulan saves a new version later, revision 1 is set to `is_active = FALSE` and revision 2 is inserted. Every downstream table — `process_instance`, `model_parameter`, `import_job`, `export_job`, `validation_run`, `bw_mapping_selection` — references `revision_id` as a FK, making the revision the central anchor of all work in the system.

```sql
INSERT INTO battery_model_revision
  (revision_id, model_id, revision_number, label,
   is_active, status, created_by, created_at, frozen_at, notes)
VALUES (
  'rev-0001-uuid',
  'mdl-0001-uuid',    -- FK -> battery_model
  1,
  'Initial import from SI2.xlsx',
  TRUE,               -- only active revision for this model
  'draft',
  'usr-Foulan-uuid',     -- FK -> app_user
  NOW(),
  NULL,               -- not frozen yet; Foulan can still edit
  'First pass import from Ali_2025_Parametric_LCA_battery_SI2.xlsx'
);
```

revision_id (PK)	model_id (FK)	rev_no	is_active	status	frozen_at	notes
rev-0001-uuid	mdl-0001-uuid	1	TRUE	draft	NULL	Initial import

The FK chain from revision back to project

`battery_model_revision.model_id` → `battery_model.model_id` → `battery_model.project_id` → `project.project_id`. This three-hop chain is how RLS traces ownership: a revision belongs to a model which belongs to a project which has `project_member` rows. Supabase's RLS policy on `battery_model_revision` joins through this chain to confirm `auth.uid()` is a member of the project before allowing any read or write.

5 Step 4 — Data Entry: Two Paths to the Same Tables

Two equally valid paths

Once a revision exists, Foulan needs to populate it with processes and exchanges. The schema supports two paths that produce **identical database rows** — the same `process_instance`, `process_exchange`, and `model_parameter` rows result either way. The difference is only in *who* creates them and *how*.

Path A — Manual entry
Foulan types data directly into the UI. Each process and exchange is created row by row via the frontend calling the Supabase JS client. No `artifact` or `import_job` row is created.

Path B — File import
Foulan uploads a pre-formatted xlsx. The FastAPI parser reads the file and bulk-inserts all rows automatically. An `artifact` and `import_job` row track the operation.

5.1 Path A — Manual Entry

STEP 4A — Foulan enters processes and exchanges directly in the UI

↓

5.1.1 model_parameter — Set by Foulan in the Parameters Panel

What it is and who creates it

`model_parameter` stores named scalar values that can be referenced by name in exchange formulas (e.g. a formula `Cor * 0.175` uses the `Cor` parameter). In the manual entry path, Foulan creates these herself in a **Parameters** panel before or during data entry. he gives each parameter a name, a central value, and optionally min/max/mode values for stochastic analysis. The UI calls the Supabase JS client directly — no FastAPI involvement. Parameters are optional; a revision with no `model_parameter` rows is perfectly valid if all exchange quantities are plain numbers.

```sql
-- Foulan types these into the Parameters panel in the UI.
-- The frontend calls supabase.from('model_parameter').insert(...)
INSERT INTO model_parameter
  (param_id, revision_id, name, description, value,
   param_type, min_value, max_value, mode_value, distribution_type)
VALUES
  ('prm-001', 'rev-0001-uuid',  -- FK -> battery_model_revision
   'Cor', 'Cobalt recovery rate',
   0.97, 'scalar', 0.95, 0.99, 0.97, 'triangle'),

  ('prm-002', 'rev-0001-uuid',
   'EEp', 'Electricity emission factor (kg CO2/kWh)',
   0.30, 'scalar', 0.05, 0.72, 0.30, 'triangle'),

  ('prm-003', 'rev-0001-uuid',
   'Librine', 'Lithium from brine share',
   0.50, 'scalar', 0.30, 0.70, NULL, 'linear');
```

param_id (PK)	revision_id (FK)	name	value	min	max	dist	description
prm-001	rev-0001-uuid	Cor	0.97	0.95	0.99	triangle	Cobalt recovery
prm-002	rev-0001-uuid	EEp	0.30	0.05	0.72	triangle	Electricity EF
prm-003	rev-0001-uuid	Librine	0.50	0.30	0.70	linear	Li from brine

UNIQUE(revision_id, name) prevents duplicate parameter names within a revision.

5.1.2 process_instance — Foulan Creates a Process

What it is

Each foreground activity is one `process_instance` row. In the manual path Foulan clicks **Add Process**, fills in the name, location, unit, and production amount, and optionally selects a template. The frontend inserts the row directly. One `process_instance` corresponds to one Brightway `Activity` node at export time. `system_boundary = 'foreground'` for all processes Foulan enters manually. Ecoinvent-imported background processes are inserted with `system_boundary = 'background'`. This field is critical for the BW export layer, which must distinguish foreground nodes (parameterisable, editable) from background nodes (fixed, resolved via ecoinvent lookup).

```sql
-- Foulan fills in the New Process form and clicks Save.
INSERT INTO process_instance
  (process_id, revision_id, name, location, unit,
   production_amount, production_unit, stage,
   template_id, system_boundary, comment, created_at)
VALUES (
  'pi-001-uuid',
  'rev-0001-uuid',            -- FK -> battery_model_revision
  'Active Cathode Material',
  'DE',                       -- selected from region_catalog dropdown
  'kg',
  1.0,                        -- production amount: almost always 1
  'kg',
  'manufacturing',
  'tpl-cathode-uuid',         -- FK -> process_template (optional; Foulan
      chose a template)
  'foreground',               -- system_boundary: 'foreground' | 'background'
  'NMC811 cathode synthesis, plant data 2024',
  NOW()
);
```

process_id (PK)	revision_id (FK)	name	loc	unit	prod_amt	sys_boundary
pi-001-uuid	rev-0001-uuid	Active Cathode Material	DE	kg	1.0	foreground

5.1.3 process_exchange — Foulan Enters Exchanges Row by Row

What it is

`process_exchange` is the densest table. Each row is one input or output of a process. In the manual path Foulan uses the exchange grid (AG Grid) to add rows one at a time. For each row he selects a flow from the `flow_catalog` dropdown (which auto-creates a new flow if the name is not found), enters a quantity or formula, selects a unit, and sets direction. There is no FastAPI involvement — the frontend writes directly to Supabase. **Manual entry does not produce** `formula_user`, `source_database`, or `source_location` values automatically; Foulan must fill these in explicitly if he wants them.

```sql
-- Foulan types "1.0 kg" in the reference flow row and selects
    direction=output.
INSERT INTO process_exchange
  (exchange_id, process_id, flow_id, raw_name,
   quantity_user, formula_user, user_unit,
   exchange_direction, output_type,
   source_database, source_location,
   amount_is_ecoinvent_signed, sort_order)
VALUES (
  'exc-001-uuid',
  'pi-001-uuid',            -- FK -> process_instance
  'flw-acm-uuid',           -- FK -> flow_catalog (selected from dropdown)
  'Active Cathode Material', -- raw_name: what Foulan typed before flow
      resolved
  1.0,
  NULL,     -- no formula; Foulan entered a plain number
  'kg',
  'output',
  'reference',              -- output_type: 'reference' | 'coproduct' |
      'waste_output' | 'stock'
  NULL,     -- source_database: Foulan can fill this in optionally
  NULL,     -- source_location: optional
  FALSE,    -- amount_is_ecoinvent_signed: FALSE for all
      manufacturer-entered data
  1
);
```

```sql
-- Foulan types "=Cor * 0.175" in the quantity field for cobalt sulfate.
-- The UI evaluates the formula using the current Cor value (0.97) to get
    0.16975,
-- stores both the evaluated result and the raw formula string.
INSERT INTO process_exchange
  (exchange_id, process_id, flow_id, raw_name,
   quantity_user, formula_user, user_unit,
   exchange_direction, output_type,
   amount_is_ecoinvent_signed, sort_order)
VALUES (
  'exc-003-uuid',
  'pi-001-uuid',
  'flw-coso4p-uuid',        -- FK -> flow_catalog: cobalt_sulfate_primary
  'cobalt sulfate',
  0.16975,                  -- evaluated: Cor(0.97) * 0.175
  '=Cor * 0.175',           -- raw formula preserved; references
      model_parameter.name
  'kg',
  'input',
  NULL,           -- output_type: NULL for inputs (only outputs have a type)
  FALSE, 3        -- amount_is_ecoinvent_signed: FALSE for all
      manufacturer-entered data
);
```

Resulting rows in process_exchange after both manual inserts.

exchange_id (PK)	process_id (FK)	flow_id (FK)	raw_name	qty

exc-001-uuid	pi-001-uuid	flw-acm-uuid	Active Cathode Material	1.0
exc-003-uuid	pi-001-uuid	flw-coso4p-uuid	cobalt sulfate	0.16975

formula	unit	dir	output_type	ecoinvent_signe
NULL	kg	output	**reference**	FALSE
=Cor*0.175	kg	input	NULL	FALSE

output_type is NULL for inputs; only output exchanges carry a type. source_database and source_location both NULL (not entered by Foulan). amount_is_ecoinvent_signed = FALSE for all foreground

Manual entry: foreground amounts must always be positive

In the manual path the UI must **prevent** a negative `quantity_user` from being saved for foreground (manufacturer-entered) processes. The UI shows a validation error and asks Foulan to enter a positive value, with directionality carried by the `exchange_direction` field. This matches the ISO 14040/14044 convention: all flow magnitudes are positive; direction is a separate attribute.
Negative amounts are only valid for ecoinvent-imported exchanges (rows where `amount_is_ecoinvent_signed = TRUE`), which represent intentional LCA modelling conventions (waste inputs, avoided burdens, soil metal uptake). These never appear in manually entered foreground data.

No artifact or import_job in the manual path

Manual entry creates **no** `artifact` row and **no** `import_job` row. Those tables only exist to track file-based operations. The absence of an `import_job` row for a revision is a normal and valid state — it simply means the data was entered by hand.

5.2 Path B — File Import

STEP 4B — Foulan uploads Foulan_2025_Parametric_LCA_battery_SI2.xlsx

↓

How Path B differs from Path A

The end result is the same rows in `process_instance`, `process_exchange`, and `model_parameter`. The difference is that a file is the source, so two additional tables are written: `artifact` (the file record) and `import_job` (the parse operation). The FastAPI parser also applies all schema fixes automatically (formula capture, source traceability, ecoinvent sign preservation) which manual entry relies on the UI to enforce.

5.2.1 artifact — The File Record

What it is

`artifact` is a unified abstraction for any file associated with a revision. When Foulan uploads his xlsx, the frontend first sends the file to **Supabase Storage**, receives back a storage path, then creates an `artifact` row. The file itself lives in object storage; the database holds only the metadata and path. This keeps binary data out of Postgres while maintaining full auditability.

```sql
INSERT INTO artifact
  (artifact_id, revision_id, artifact_type, filename,
   storage_path, mime_type, size_bytes, checksum_sha256, created_at)
VALUES (
  'art-001-uuid',
  'rev-0001-uuid',          -- FK -> battery_model_revision
  'import',                 -- artifact_type = 'import' for source files [Q8
      fix]
  'Ali_2025_Parametric_LCA_battery_SI2.xlsx',
  'projects/proj-0001/revisions/rev-0001/imports/SI2.xlsx',
  'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet',
  284672,
  'sha256:a3f8c2d1...',
  NOW()
);
```

artifact_id (PK)	revision_id (FK)	artifact_type	filename	storage_path
art-001-uuid	rev-0001-uuid	export	SI2.xlsx	.../imports/SI2.xlsx

Why checksum matters

`checksum_sha256` lets anyone verify that the file in storage matches the one Foulan uploaded. This is a legal traceability requirement for certified LCA studies.

5.2.2 import_job — Tracking the Parse Operation

What it is

`import_job` tracks the *operation* of parsing the uploaded file, separately from the `artifact` which is just the file record. The separation matters because an import can fail partway through — the artifact row exists regardless, but the `import_job` records whether parsing succeeded, how many rows were created, and any per-row errors. The frontend polls `import_job.status` to show a progress indicator to Foulan.

```sql
INSERT INTO import_job
  (import_id, revision_id, source_filename, source_format,
   imported_by, imported_at, status,
   activities_count, exchanges_count, warnings_count, errors_count,
       log_json)
VALUES (
  'imp-001-uuid',
  'rev-0001-uuid',           -- FK -> battery_model_revision
  'Ali_2025_Parametric_LCA_battery_SI2.xlsx',
  'brightway_xlsx',
  'usr-Foulan-uuid',            -- FK -> app_user
  NOW(),
  'running',
  NULL, NULL, 0, 0, NULL    -- populated on completion
);
```

The FastAPI import parser runs. It downloads the file from Supabase Storage, parses every activity, parameter, and exchange row using `openpyxl`, applies all schema fixes, and bulk-inserts into `model_parameter`, `process_instance`, and `process_exchange`. On completion it updates the `import_job` row:

```sql
UPDATE import_job SET
  status           = 'completed',
  activities_count = 83,
  exchanges_count  = 984,
  warnings_count   = 1,
  errors_count     = 0,
  log_json         = '{"flipped": 47, "formulas_captured": 312,
                       "params_created": 14, "sources_captured": 984}'
WHERE import_id = 'imp-001-uuid';
```

import_id (PK)	revision_id (FK)	status	activities	exchanges	warnings	errors
imp-001-uuid	rev-0001-uuid	completed	83	984	1	0

5.2.3 model_parameter — Created by the Parser from the File Header

What it is and who creates it in Path B

In the file import path, the parser reads the parameter table from the xlsx header (SI2 Table 1) and creates `model_parameter` rows automatically. Foulan does not type these in. After import he sees them in the Parameters panel and can review or adjust values — but the rows were created by the parser, not by his directly. This is the key difference from Path A where he creates them herself.

```
-- Parser reads SI2 Table 1 rows and inserts one model_parameter per
   parameter found.
INSERT INTO model_parameter
  (param_id, revision_id, name, description, value,
   param_type, min_value, max_value, mode_value, distribution_type)
VALUES
  ('prm-001', 'rev-0001-uuid', 'Cor', 'Cobalt recovery rate',
   0.97, 'scalar', 0.95, 0.99, 0.97, 'triangle'),
  ('prm-002', 'rev-0001-uuid', 'EEp', 'Electricity emission factor kg
     CO2/kWh',
   0.30, 'scalar', 0.05, 0.72, 0.30, 'triangle'),
  ('prm-003', 'rev-0001-uuid', 'Librine', 'Lithium from brine share',
   0.50, 'scalar', 0.30, 0.70, NULL, 'linear');
-- ... 11 more: Alr, Grr, Cur, Lir, Mnr, Nir, NiNi_NiCo, Sp, CRp, SEbp, REg
```

param_id (PK)	revision_id (FK)	name	value	min	max	dist_type	description
prm-001	rev-0001-uuid	Cor	0.97	0.95	0.99	triangle	Cobalt recovery
prm-002	rev-0001-uuid	EEp	0.30	0.05	0.72	triangle	Electricity EF
prm-003	rev-0001-uuid	Librine	0.50	0.30	0.70	linear	Li from brine
...11 more	rev-0001-uuid						

14 rows total. UNIQUE(revision_id, name) enforced.

5.2.4 process_instance and process_exchange in Path B

In Path B the parser creates `process_instance` and `process_exchange` rows with the same structure as Path A. The differences are:

Field	Path A (manual)	Path B (import)
`process_instance.comment`	Foulan types a free-text note	Parser writes `"Imported from SI2 row N"`
`process_instance.system_boundary`	UI sets `'foreground'` always	Parser sets `'background'` for ecoinvent-sourced rows; `'foreground'` for manufacturer rows

`process_exchange.raw_name`	Whatever Foulan typed before flow resolved	Verbatim string from the xlsx cell
`process_exchange.output_type`	UI sets from dropdown (`reference` / `coproduct` / `waste_output` / `stock`)	Parser reads from spold `outputGroup` attribute directly
`process_exchange.formula_user`	Foulan types the formula explicitly if he wants it	Parser captures every non-trivial formula string for audit; never re-evaluated [Q14]
`process_exchange.source_database`	Foulan fills in optionally	Parser extracts from the SI2 source column automatically
`process_exchange.amount_is_ecoinvent_signed`	Auto-assigned FALSE	TRUE for ecoinvent `.spold` sources; FALSE for manufacturer SI2 sources
Negative `quantity_user`	UI prevents negative entry; `NEGATIVE_FOREGROUND_QTY` validation rule fires if one slips through	Parser preserves ecoinvent negatives as-is (sets `amount_is_ecoinvent_signed = TRUE`); flags manufacturer negatives via the same rule

The SQL for these rows is otherwise identical to the exchange scenarios shown in the process_exchange section that follows.

process_exchange — The Core Inventory Records (Both Paths)

What it is

`process_exchange` is the densest table. Each row is one quantified input or output of a foreground process. Whether created by hand (Path A) or by the parser (Path B), the row structure is identical. The following scenarios illustrate the exchange types that can arise.

5.2.5 Scenario A — Normal Technosphere Input

A standard input such as *nickel_sulfate* (1.54 kg, plain numeric, from `NMC811_parametric_model`):

INSERT: process_exchange — normal input

```
INSERT INTO process_exchange
  (exchange_id, process_id, flow_id, raw_name,
   quantity_user, formula_user, user_unit,
   exchange_direction, output_type,
   source_database, source_location,
   amount_is_ecoinvent_signed, sort_order)
VALUES (
  'exc-002-uuid', 'pi-001-uuid',
  (SELECT flow_id FROM flow_catalog WHERE
      canonical_name='nickel_sulfate_primary'),
  'nickel_sulfate',             -- raw_name verbatim from SI2
```

```
  1.54,                          -- quantity_user: pre-computed amount used
      as-is
  '=1.54',                       -- formula_user: raw string captured for
      audit only [Q14]
  'kg',
  'input',                       -- exchange_direction: standard input
  NULL,                          -- output_type: NULL for inputs
  'NMC811_parametric_model',     -- source_database
  'DE',                          -- source_location
  FALSE, 2                       -- amount_is_ecoinvent_signed: FALSE
      (manufacturer data)
);
```

5.2.6 Scenario B — Negative Amount from Ecoinvent (Preserved As-Is)

A wastewater treatment exchange sourced from ecoinvent arrives with a negative amount. Unlike the old FIX-01 rule (which would have flipped this to a positive output), the parser now stores it exactly as found in the source file. The `amount_is_ecoinvent_signed` flag tells the BW export layer to apply the correct A-matrix sign convention at export time.

In this example the exchange comes from a manufacturer's own SI2 file, not from an ecoinvent `.spold` file, so `amount_is_ecoinvent_signed = FALSE` and the negative value will trigger the `NEGATIVE_FOREGROUND_QTY` validation rule — Foulan should correct his formula. If this were an ecoinvent-sourced exchange, the flag would be TRUE and no validation error would fire.

```
INSERT INTO process_exchange
  (exchange_id, process_id, flow_id, raw_name,
   quantity_user, formula_user, user_unit,
   exchange_direction, output_type,
   source_database, source_location,
   amount_is_ecoinvent_signed, sort_order)
VALUES (
  'exc-009-uuid', 'pi-001-uuid',
  (SELECT flow_id FROM flow_catalog WHERE
      canonical_name='market_wastewater_avg'),
  'market for wastewater, average',
  -0.0675,                -- stored exactly as entered; NOT flipped
  '=-67.5/1000',          -- formula_user: raw string for audit [Q14]
  'm3',
  'input',                -- direction not changed; Foulan must correct this
  NULL,                   -- output_type: NULL for inputs
  'NMC811_parametric_model',
  'DE',
  FALSE,                  -- FALSE = manufacturer foreground;
      NEGATIVE_FOREGROUND_QTY rule fires
  14
);
-- Validation engine will raise NEGATIVE_FOREGROUND_QTY on this row.
-- Foulan should change quantity to 0.0675 and exchange_direction to
    'output',
-- OR change the formula to output a positive value.
```

5.2.7 Scenario C — Biosphere (Elementary) Exchange

The COD emission to water from *Styrene butadiene rubber*:

```sql
INSERT INTO process_exchange
  (exchange_id, process_id, flow_id, raw_name,
   quantity_user, formula_user, user_unit,
   exchange_direction, output_type,
   source_database, source_location,
   amount_is_ecoinvent_signed, sort_order)
VALUES (
  'exc-bio-001', 'pi-003-uuid',
  (SELECT flow_id FROM flow_catalog
   WHERE canonical_name='COD_chemical_oxygen_demand'),
  'COD, Chemical Oxygen Demand',
  0.000175,
  '=1.75/10000',       -- formula_user: stored for audit; amount already
      evaluated [Q14]
  'kg',
  'output',            -- biosphere emissions always direction=output
  NULL,                -- output_type: NULL (biosphere; output_type applies
      to technosphere outputs only)
  'biosphere3',        -- source_database
  NULL,                -- biosphere flows have no location
  FALSE, 11            -- amount_is_ecoinvent_signed: FALSE (foreground
      process)
);
```

5.2.8 Scenario D — Production (Reference) Flow

Every activity must have exactly one exchange with `is_reference_flow = TRUE`. This is the defining output:

```sql
INSERT INTO process_exchange
  (exchange_id, process_id, flow_id, raw_name,
   quantity_user, formula_user, user_unit,
   exchange_direction, output_type,
   source_database, source_location,
   amount_is_ecoinvent_signed, sort_order)
VALUES (
  'exc-001-uuid', 'pi-001-uuid',
  (SELECT flow_id FROM flow_catalog
   WHERE canonical_name='Active_Cathode_Material'),
  'Active Cathode Material',
  1.0,
  '=1',
  'kg',
  'output',          -- reference product outputs are always outputs
  'reference',       -- output_type = 'reference' (identifies this as the
      defining output)
  'NMC811_parametric_model',
  'DE',
  FALSE,             -- amount_is_ecoinvent_signed: FALSE (foreground data)
  1
);
```

raw_name	qty	dir	output_type	ecoinvent_si	note

Active Cathode Material	1.0 kg	output	**reference**	FALSE	Reference product
nickel_sulfate	1.54 kg	input	NULL	FALSE	Formula captured [Q14]
cobalt_sulfate	0.175 kg	input	NULL	FALSE	Formula captured
electricity	6.21 kWh	input	NULL	FALSE	Formula `=0.00621*1000`
market for wastewater	−0.0675 m3	input	NULL	FALSE	Negative; NEGATIVE_FOREGROUND_QTY fires
COD Chemical Oxygen Demand	0.000175 kg	output	NULL	FALSE	Biosphere; output_type NULL

5.3 process_link — Explicit Graph Edges (Optional)

What it is

`process_link` stores explicit directed edges between processes in the foreground graph. In most cases the export engine can *infer* links by matching exchange names (e.g. the `Cathode` process has an input named `Active Cathode Material` which matches the output of the `Active Cathode Material` process). Explicit links are written when the inference would be ambiguous — for example, if two processes both output `electricity` and a third process consumes it.

```sql
INSERT INTO process_link
  (link_id, revision_id, from_process_id, to_process_id, flow_id)
VALUES (
  'lnk-001-uuid',
  'rev-0001-uuid',
  'pi-001-uuid',    -- Active Cathode Material (producer)
  'pi-002-uuid',    -- Cathode (consumer)
  (SELECT flow_id FROM flow_catalog
   WHERE canonical_name = 'Active_Cathode_Material')
);
```

6 Step 5 — Validation Run

STEP 5 — Validation runs automatically in the background after every save

↓

Validation is continuous — not a manual step

Validation runs automatically in the background after every save, debounced by 3 seconds after the last edit. The frontend calls the `trigger-validation` Edge Function silently — Foulan does not need to remember to click a button. A validation badge on each process in the sidebar shows pass, warning, or error status in real time. A "Re-validate" button is available for manual reassurance but is not required. This design choice was made explicitly because manufacturers are assumed to be lazy about manual validation steps: running validation automatically ensures errors are surfaced immediately rather than at export time when fixing them is more disruptive.
Full-revision validation always runs (all 83 processes, all 984 exchanges) — this is simple to implement and fast enough at this scale. The `validation_run` table records every run; only the most recent run per revision is shown in the UI.

The `trigger-validation` Edge Function queues a FastAPI validation job. The validation engine runs all rules from `validation_rule` against the current revision data and writes results to `validation_run` and `validation_issue`.

6.1 validation_run — The Snapshot Record

```sql
INSERT INTO validation_run
  (validation_id, revision_id, triggered_by, run_at,
   status, issue_count, tool_version, config_hash)
VALUES (
  'val-001-uuid',
  'rev-0001-uuid',
  'usr-Foulan-uuid',
  NOW(),
  'pass',     -- updated after all rules run
  5,          -- 0 errors, 4 info/warn items
  '1.2.0',
  'sha256:c4e7f...'  -- hash of validation_rule rows used
);
```

Why config_hash matters

If the validation rules change (e.g. a new rule is added to `validation_rule`), re-running validation on the same revision will produce a different `config_hash`. This makes it auditable which rule set was active when a revision was validated — important for certified LCA studies that require documented validation provenance.

6.2 validation_issue — One Row Per Finding

The engine runs each rule from `validation_rule` against the revision data. Here are both scenarios: a passing run and a failing one.

6.2.1 Scenario A — Validation Passes (Informational Items Only)

```sql
-- Rule: FLIP_APPLIED fired for every flipped exchange
INSERT INTO validation_issue
  (issue_id, validation_id, severity, code, message,
   process_id, exchange_id, suggestion)
VALUES
  ('iss-001', 'val-001-uuid', 'info', 'ECOINVENT_NEGATIVE_AMT',
   'exc-042: nickel_sulfate from ecoinvent has quantity_user=-0.003 '
   '(amount_is_ecoinvent_signed=TRUE). Intentional; no action required.',
   'pi-004-uuid', 'exc-042-uuid', NULL),

  ('iss-002', 'val-001-uuid', 'info', 'FORMULA_PRESERVED',
   'exc-008 (electricity): formula_user=''=0.00621*1000'' stored.
      Parametric substitution enabled.',
   'pi-001-uuid', 'exc-008-uuid', NULL),

  ('iss-003', 'val-001-uuid', 'warning', 'UNMAPPED_FLOW',
   'flow Aerophine: mapping_status=pending. Export will be blocked until
      mapped.',
   'pi-075-uuid', NULL,
   'Map to an ecoinvent equivalent or set mapping_status=unmappable to
      proceed.'),

  ('iss-004', 'val-001-uuid', 'info', 'FOREGROUND_CONFIRMED',
   '9 flows have mapping_status=foreground. No background lookup will be
      attempted.',
   NULL, NULL, NULL),

  ('iss-005', 'val-001-uuid', 'info', 'PARAM_STOCHASTIC',
   '14 model_parameter rows have distribution_type set. Monte Carlo
      enabled.',
   NULL, NULL, NULL);
```

code	severity	entity	message (abbreviated)
ECOINVENT_NEGATIVE_AMT	info	exc-042	ecoinvent negative amount preserved; intentional
FORMULA_PRESERVED	info	exc-008	formula =0.00621*1000 captured
UNMAPPED_FLOW	warning	pi-075	Aerophine not yet mapped
FOREGROUND_CONFIRMED	info	(revision)	9 foreground flows confirmed
PARAM_STOCHASTIC	info	(revision)	14 parameters with distributions

Result: validation_run.status

status = 'pass' because there are zero error-severity issues. The one warning (Aerophine unmapped) does not block export — it will produce a warning in the export log. Foulan can proceed to BW mapping.

6.2.2 Scenario B — Validation Fails (Error Found)

Suppose Foulan has manually entered a negative quantity (-1.54 kg for nickel sulfate) without flagging it as ecoinvent-sourced. The NEGATIVE_FOREGROUND_QTY rule fires, because amount_is_ecoinvent_signe = FALSE on this row:

```sql
INSERT INTO validation_issue
  (issue_id, validation_id, severity, code, message,
   process_id, exchange_id, suggestion)
VALUES (
  'iss-err-001', 'val-002-uuid', 'error', 'NEGATIVE_FOREGROUND_QTY',
  'exc-002 (nickel_sulfate): quantity_user = -1.54 is negative but '
  'amount_is_ecoinvent_signed = FALSE. Foreground (manufacturer-entered) '
  'exchange magnitudes must be positive; direction is carried by
     exchange_direction.',
  'pi-001-uuid', 'exc-002-uuid',
  'Change quantity_user to 1.54 and set exchange_direction = ''output'' if
     this '
  'is a waste output. If this value was imported from ecoinvent, set '
  'amount_is_ecoinvent_signed = TRUE and the rule will not fire.'
);
```

```sql
UPDATE validation_run
SET status = 'fail', issue_count = 1
WHERE validation_id = 'val-002-uuid';
```

Result: validation_run.status

`status = 'fail'` because at least one `error`-severity issue exists. The export button in the UI is disabled. The `validation_issue` row is linked to `exchange_id = 'exc-002-uuid'`, so the UI highlights the exact row in the AG Grid exchange table that needs fixing.

6.2.3 Scenario C — Missing Reference Flow

A process has no exchange with `is_reference_flow = TRUE`:

```sql
INSERT INTO validation_issue
  (issue_id, validation_id, severity, code, message,
   process_id, exchange_id, suggestion)
VALUES (
  'iss-err-002', 'val-003-uuid', 'error', 'MISSING_REF_FLOW',
  'process pi-005 (Cell Container and Lid): no exchange has '
  'output_type = ''reference''. Every process must declare its reference
     product.',
  'pi-005-uuid', NULL,
  'Set output_type = ''reference'' on one output exchange for this process.
     '
  'Every process must declare exactly one reference product output.'
);
```

7 Step 6 — Brightway Mapping

STEP 6 — Foulan reviews and confirms flow mappings

↓

BW mapping is the step that links each foreground flow to either (a) a background Brightway dataset in ecoinvent, or (b) an internal foreground process, or (c) marks it as unmappable. It uses three tables: `mapping_job` (tracks async candidate generation), `bw_mapping_candidate` (automated suggestions), and `bw_mapping_selection` (Foulan's confirmed choices).

7.1 mapping_job — Async Candidate Generation Tracker

What it is and when it runs

When an import job completes, its completion webhook automatically fires the FastAPI `/candidates` endpoint as a background task. TF-IDF fuzzy matching against the full ecoinvent index for 493 flows can take 30–120 seconds, so this runs asynchronously — Foulan does not wait for it. A `mapping_job` row is created immediately, mirroring the `import_job` pattern: the frontend polls `mapping_job.status` and shows a "Generating suggestions..." spinner on the mapping screen until `status = completed`. If Foulan adds new exchanges after the initial import, he can trigger re-generation manually; a new `mapping_job` row is created for each run.

```sql
INSERT INTO mapping_job
  (mapping_job_id, revision_id, triggered_by_import_id,
   status, flow_count, matched_count, generated_at, completed_at, log_json)
VALUES (
  'mpj-001-uuid',
  'rev-0001-uuid',          -- FK -> battery_model_revision
  'imp-001-uuid',           -- FK -> import_job that triggered this
  'running',
  NULL, NULL,               -- populated on completion
  NOW(),
  NULL, NULL
);
```

```sql
UPDATE mapping_job SET
  status        = 'completed',
  flow_count    = 493,
  matched_count = 441,     -- 52 flows returned no candidates above threshold
  completed_at  = NOW(),
  log_json      = '{"unmatched": ["Aerophine", "Flotanol C07", ...]}'
WHERE mapping_job_id = 'mpj-001-uuid';
```

mapping_job_id	revision_id (FK)	status	flows	matched	completed_at

mpj-001-uuid	rev-0001-uuid	completed	493	441	2024-01-10 09:45

7.2 bw_mapping_candidate — Automated Suggestions

What it is

The FastAPI `/candidates` endpoint fuzzy-matches each `flow_catalog` entry against the ecoinvent activity index and writes candidate rows ranked by `match_score`. Multiple candidates per flow are normal — Foulan picks the right one. Candidates are never deleted; they remain for audit.

```sql
INSERT INTO bw_mapping_candidate
  (candidate_id, flow_id, bw_database, bw_activity_key,
   bw_activity_name, bw_location, bw_unit,
   match_score, match_method, generated_at)
VALUES
  -- Best match (score 0.94)
  ('cand-001', 'flw-cobalt-primary-uuid',
   'ecoinvent-3.9.1-cutoff',
   '(ecoinvent-3.9.1-cutoff, a3c2e1f9)',
   'market for cobalt sulfate',
   'GLO', 'kg',
   0.94, 'fuzzy_tfidf', NOW()),

  -- Second match (score 0.71)
  ('cand-002', 'flw-cobalt-primary-uuid',
   'ecoinvent-3.9.1-cutoff',
   '(ecoinvent-3.9.1-cutoff, b4d3f2a1)',
   'cobalt sulfate production, from sulfuric acid leaching',
   'RER', 'kg',
   0.71, 'fuzzy_tfidf', NOW());
```

cand_id	bw_activity_name	location	score	method
cand-001	market for cobalt sulfate	GLO	**0.94**	fuzzy_tfidf
cand-002	cobalt sulfate production, from H_2SO_4 leaching	RER	0.71	fuzzy_tfidf

Scenario: No candidates found (Aerophine)

For the custom reagent *Aerophine*, fuzzy matching returns no results above the minimum threshold score (0.3). The API returns an empty candidate list. Foulan must either find a proxy manually or declare it unmappable.

7.3 bw_mapping_selection — Foulan's Confirmed Choices

What it is

`bw_mapping_selection` is the *authoritative* mapping table. One row per flow per revision. The `mapping_status` enum is the key discriminator:

- **mapped** — Foulan selected a `bw_mapping_candidate` and confirmed it
- **foreground** — the flow is itself a foreground process; resolved internally
- **unmappable** — no BW equivalent; exchange will be excluded from export
- **pending** — awaiting decision; blocks export

The `UNIQUE(flow_id, revision_id)` constraint ensures Foulan cannot create two conflicting mappings for the same flow.

7.3.1 Scenario A — Foulan confirms a background mapping

```
INSERT INTO bw_mapping_selection
  (mapping_id, flow_id, revision_id, candidate_id,
   mapping_status, confirmed_by, confirmed_at, notes)
VALUES
  -- cobalt_sulfate_primary: Foulan picks cand-001
  ('map-001', 'flw-cobalt-primary-uuid', 'rev-0001-uuid',
   'cand-001',    -- points to the chosen candidate
   'mapped',      -- confirmed background BW dataset
   'usr-Foulan-uuid', NOW(),
   'Best match confirmed. GLO location appropriate for global supply
       chain.'),

  -- nickel_sulfate_primary: mapped to NiSO4 activity
  ('map-002', 'flw-nickel-primary-uuid', 'rev-0001-uuid',
   'cand-010',
   'mapped',
   'usr-Foulan-uuid', NOW(), NULL),

  -- market_wastewater_avg: mapped to ecoinvent
  ('map-003', 'flw-wastewater-uuid', 'rev-0001-uuid',
   'cand-020',
   'mapped',
   'usr-Foulan-uuid', NOW(), NULL);
```

7.3.2 Scenario B — Foreground flow (no background lookup)

INSERT: bw_mapping_selection — foreground flows (FIX-09)

```
INSERT INTO bw_mapping_selection
  (mapping_id, flow_id, revision_id, candidate_id,
   mapping_status, confirmed_by, confirmed_at, notes)
VALUES
  -- electricity: is itself a foreground process_instance (pi-027-uuid)
  -- The export engine resolves this via process_instance.name matching,
  -- NOT via an ecoinvent lookup. candidate_id is NULL.
  ('map-004', 'flw-electricity-uuid', 'rev-0001-uuid',
   NULL,            -- no candidate; foreground resolution
   'foreground',    -- FIX-09: explicitly marks as internal
   'usr-Foulan-uuid', NOW(),
```

```
    'Resolves to process_instance pi-027-uuid (electricity). '
    'No ecoinvent lookup. Prevents double-counting.'),

  -- Active Cathode Material: foreground process pi-001-uuid
  ('map-005', 'flw-acm-uuid', 'rev-0001-uuid',
   NULL, 'foreground', 'usr-Foulan-uuid', NOW(),
   'Resolves to process_instance pi-001-uuid.');
```

7.3.3 Scenario C — Unmappable flow

```
INSERT INTO bw_mapping_selection
  (mapping_id, flow_id, revision_id, candidate_id,
   mapping_status, confirmed_by, confirmed_at, notes)
VALUES (
  'map-006', 'flw-aerophine-uuid', 'rev-0001-uuid',
  NULL,
  'unmappable',     -- intentionally excluded from export
  'usr-Foulan-uuid', NOW(),
  'Custom flotation reagent with no ecoinvent equivalent. '
  'Excluded from BW export. Characterisation factor to be added manually '
  'in Brightway after import.'
);
```

flow (canonical_name)	status	confirmed_by	note
`cobalt_sulfate_primary`	mapped	Foulan Foulani	Best match; GLO location confirmed
`nickel_sulfate_primary`	mapped	Foulan Foulani	
`market_wastewater_avg`	mapped	Foulan Foulani	
`COD_chem_oxygen_demand`	mapped	Foulan Foulani	biosphere3 dataset
electricity	foreground	Foulan Foulani	Resolves to pi-027; no BW lookup
Active_Cathode_Material	foreground	Foulan Foulani	Resolves to pi-001
`Aerophine`	unmappable	Foulan Foulani	No ecoinvent equivalent
`lithium_hydroxide_primary`	pending	—	Not yet reviewed

pending mappings block export

Any `mapping_status = 'pending'` row will cause the export engine to raise an error before generating any file. The UI shows a count of pending mappings and highlights the affected flows in the mapping table. Foulan must resolve all pending mappings before clicking Export.

8 Step 7 — Export

STEP 7 — Foulan clicks "Export to Brightway"

↓

Export involves the same three table types as import but in the opposite direction: `export_job` tracks the operation, the FastAPI service generates the file, and `artifact` stores the result.

8.1 export_job — Tracking the Export Operation

What it is

`export_job` tracks each export operation independently from `artifact`. The job record exists from the moment Foulan clicks Export (`status = 'running'`) and is updated on success or failure. The `artifact_id` FK is populated once the file has been written to Supabase Storage. This separation means a failed export (e.g. unmapped flows found) has a `export_job` record showing what went wrong but no `artifact` row (no file was produced).

8.1.1 Scenario A — Export Succeeds

```
INSERT INTO export_job
  (export_id, revision_id, format, exported_by, exported_at,
   status, activities_exported, exchanges_exported, artifact_id, error_log)
VALUES (
  'exp-001-uuid',
  'rev-0001-uuid',
  'brightway_xlsx',
  'usr-Foulan-uuid',
  NOW(),
  'running',
  NULL, NULL, NULL, NULL  -- populated on completion
);
```

The FastAPI service:

1. Loads all 83 `process_instance` rows and 984 `process_exchange` rows.
2. Checks that all `bw_mapping_selection.mapping_status` values are `mapped`, `foreground`, or `unmappable` (none `pending`).
3. Builds the Brightway Excel workbook using `openpyxl`: one sheet per activity.
4. For `foreground` flows: writes the internal activity name as the link target.
5. For `mapped` flows: writes the `bw_activity_key` from the selected `bw_mapping_candidate`.
6. For `unmappable` flows: skips the exchange and logs a warning.
7. Uploads the file to Supabase Storage; creates an `artifact` row; updates `export_job`.

INSERT: artifact (export file)

```
INSERT INTO artifact
  (artifact_id, revision_id, artifact_type, filename,
   storage_path, mime_type, size_bytes, checksum_sha256, created_at)
VALUES (
  'art-002-uuid',
  'rev-0001-uuid',
  'export',
```

```
  'Ali_2025_NMC811_bw_export_rev1.xlsx',
  'projects/proj-0001/revisions/rev-0001/exports/bw_rev1.xlsx',
  'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet',
  512000,
  'sha256:f9e3b2...',
  NOW()
);
```

```
UPDATE export_job SET
  status                = 'completed',
  activities_exported   = 83,
  exchanges_exported    = 980,   -- 984 minus 4 Aerophine exchanges
      (unmappable)
  artifact_id           = 'art-002-uuid',
  error_log             = NULL
WHERE export_id = 'exp-001-uuid';
```

Export completed — artifact created

The frontend receives the completed `export_job` and requests a signed download URL from Supabase Storage for `art-002-uuid`. Foulan downloads the Brightway xlsx. The `checksum_sha256` on the artifact allows anyone to verify the file integrity later.

8.1.2 Scenario B — Export Fails (Pending Mappings)

If Foulan had not finished mapping all flows:

```
UPDATE export_job SET
  status    = 'failed',
  error_log = 'Export aborted: 2 flows have mapping_status=pending. '
              'Pending flows: lithium_hydroxide_primary (pi-001, pi-038), '
              'manganese_sulfate_primary (pi-001). '
              'Resolve all pending mappings before re-exporting.'
WHERE export_id = 'exp-002-uuid';
-- No artifact row is created when export fails.
```

No artifact created on failure

`export_job.artifact_id` remains NULL. No file was written to storage. The error_log tells Foulan exactly which flows to map. The export_job row is preserved for audit — it shows that an export was attempted at this timestamp, why it failed, and who triggered it.

8.2 Freezing the Revision After Export

Once Foulan is satisfied with the export, he freezes the revision to make it immutable:

```sql
UPDATE battery_model_revision SET
  status    = 'frozen',
  frozen_at = NOW()
WHERE revision_id = 'rev-0001-uuid';
```

frozen_at prevents further edits

Once `frozen_at` is set, the application layer (FastAPI + RLS policy) blocks any `INSERT`, `UPDATE`, or `DELETE` on `process_instance`, `process_exchange`, or `model_parameter` for this revision.

8.3 Admin Unfreeze — Recovery Path

Unfreezing is admin-only and fully audited

If a critical error is found after freezing, only an admin can unfreeze the revision. The `battery_model_revision` table carries an `unfreeze_log` JSONB column (array of log entries) that records every unfreeze event. No separate table is needed — the JSONB array on the revision row is sufficient for audit purposes. Manufacturers cannot self-serve unfreeze.

```sql
-- Added to battery_model_revision at migration time:
ALTER TABLE battery_model_revision
  ADD COLUMN unfreeze_log JSONB DEFAULT '[]'::jsonb;
-- Each unfreeze event appends one object to the array:
-- {"unfrozen_by": "admin-uuid", "unfrozen_at": "2024-02-01T10:00:00Z",
--  "reason": "Incorrect wastewater formula found post-certification",
--  "previously_frozen_at": "2024-01-20T14:30:00Z"}
```

```sql
-- Called via the admin panel. Requires service-role key.
-- Verifies caller has role = admin before proceeding.
UPDATE battery_model_revision SET
  status      = 'draft',
  frozen_at   = NULL,
  unfreeze_log = unfreeze_log || jsonb_build_array(jsonb_build_object(
    'unfrozen_by',          'admin-user-uuid',
    'unfrozen_at',          NOW(),
    'reason',               'Incorrect wastewater formula found post-export',
    'previously_frozen_at', frozen_at
  ))
WHERE revision_id = 'rev-0001-uuid';
-- After this UPDATE Foulan can edit the revision again.
-- The unfreeze_log preserves the full history permanently.
```

9 Step 8 — Monte Carlo Parametric Export

STEP 8 — Foulan runs Monte Carlo analysis (N=1000 samples)

↓

What triggers this

Foulan selects "Stochastic Export" with 1000 samples. The FastAPI `/montecarlo` endpoint reads the `model_parameter` distributions and the `process_exchange.formula_user` strings to generate N sampled LCI datasets.

How the parameter table is used. For each of 1000 iterations:

1. For each row in `model_parameter` WHERE `distribution_type IS NOT NULL`:
   - `triangle`: sample from `scipy.stats.triang(c=(mode-min)/(max-min), loc=min, scale=max-min)`
   - `linear`: sample from `numpy.random.uniform(min_value, max_value)`
2. Substitute sampled values into each `process_exchange.formula_user` string.
3. Evaluate formulas using `asteval` (safe sandboxed evaluator): `aeval = Interpreter({''Cor'': 0.968, ''EEp'': 0.31, ...}); aeval(formula_user)`. `asteval` permits only arithmetic operators, basic math functions (`abs`, `sqrt`, `min`, `max`), and named variable references. It explicitly blocks imports, attribute access, and any dangerous Python constructs. A formula that fails to parse at import time is stored with `quantity_user = NULL` and raises a `warning` (not an error).
4. Write the resulting LCI to a Brightway `wurst`-compatible JSON structure.

Artifact created.

```sql
INSERT INTO artifact
  (artifact_id, revision_id, artifact_type, filename,
   storage_path, mime_type, checksum_sha256, created_at)
VALUES (
  'art-003-uuid',
  'rev-0001-uuid',
  'parameter_set',
  'Ali_2025_NMC811_montecarlo_N1000.json',
  'projects/proj-0001/revisions/rev-0001/exports/mc_1000.json',
  'application/json',
  'sha256:d8f2c4...', NOW()
);
```

10 Complete Table-to-Step Cross-Reference

The table below summarises which steps write to which tables, making the complete data flow explicit.

Table	Step	Written by / Reason
catalog_set	Bootstrap	Admin. One-time setup. Defines the flow namespace.
unit_catalog	Bootstrap	Pre-loaded. SI conversion factors for all units.
region_catalog	Bootstrap	Pre-loaded. ISO + ecoinvent geographic codes.
data_origin_catalog	Bootstrap	Pre-loaded. Provenance categories for exchanges.
flow_catalog	Bootstrap	Pre-loaded. All 493 named flows. Central vocabulary.
flow_allowed_unit	Bootstrap	Pre-loaded. Permitted units per flow.
validation_rule	Bootstrap	Pre-loaded. Machine-readable constraint definitions.
process_template	Bootstrap	Pre-loaded. Blueprints for standard processes.
template_expected_exchange	Bootstrap	Pre-loaded. Expected flows per template.
app_user	Step 1	Edge Function `on-user-created` on invite acceptance.
project	Step 2	**Admin only.** Project created by admin via Edge Function.
project_member	Step 2	**Admin only.** Admin assigns manufacturer via Edge Function. `assigned_by` column records who made the assignment.
battery_model	Step 3	Foulan. Creates model within his admin-assigned project.
battery_model_revision	Step 3	Foulan. Draft revision; all data attached to this.
model_parameter	Step 3	Foulan (or import parser). 14 stochastic parameters.
artifact	Steps 4, 7, 8	Step 4: imported file record. Step 7: export file. Step 8: MC results.
import_job	Step 4	FastAPI parser. Tracks import operation; logs errors.
process_instance	Step 4	FastAPI parser. One row per activity (83 rows).
process_exchange	Step 4	FastAPI parser. One row per exchange (984 rows). All fixes applied.
process_link	Step 4	FastAPI parser (optional). Explicit foreground graph edges.
validation_run	Step 5	Validation engine. Snapshot of one validation execution.

validation_issue	Step 5	Validation engine. One row per finding (error/warn/info).
mapping_job	Step 6	Auto-triggered on import completion. Tracks async candidate generation.
bw_mapping_candidate	Step 6	FastAPI /candidates endpoint. Automated suggestions.
bw_mapping_selection	Step 6	Foulan. Confirmed choices: mapped/foreground/unmappable/pending.
export_job	Step 7	FastAPI exporter. Tracks export operation; links to artifact.
artifact	Step 7	FastAPI exporter. Export file record (only on success).
battery_model_revision	Post-export	Foulan. frozen_at set; revision becomes immutable.
battery_model_revision	Admin (recovery)	Admin. unfreeze-revision Edge Function sets frozen_at = NULL and appends to unfreeze_log JSONB.

11 Schema Changes from Corpus Scan Analysis (Sessions 4–6)

Summary of changes made after full ecoinvent corpus scan

After scanning all 26,533 ecoinvent .spold files and 11 XML master data files, four design questions (Q11–Q14) were resolved. The changes below update the schema from the pre-build v5 baseline. All SQL examples in this document already reflect these decisions.

11.1 Q11 — Output Type on process_exchange

Decision: Replace the planned boolean is_reference_product with an output_type ENUM column on process_exchange.

Rationale: Ecoinvent's cutoff system model distinguishes four semantically different output categories, each handled differently by allocation logic and by the BW export layer:

output_type value	spold outputGroup	Meaning
reference	0	The defining output of this unit process (the product the process is named for)
coproduct	2	An economically valuable co-product (cut off in cutoff system model; gets own .spold file)
waste_output	3	A waste stream sent to a treatment process (negative economic value)
stock	5	Material going into long-term stock (infrastructure, long-lived goods)

output_type is NULL for all input exchanges (inputGroup entries). Only outputs carry a type.

Parser behaviour: The ecoinvent import parser reads the outputGroup XML attribute directly from each exchange element and maps it to the enum value. No UUID matching is

required. For manufacturer foreground data, the UI presents a dropdown with the four values; `reference` is the default for the first output.

Battery relevance: Multi-output battery-relevant activities confirmed in the corpus include NMC/NCA/NMC hydroxide production (reference product + sodium sulfate co-product), cobalt production (22 outputs including cobalt, nickel sulfate, and wastewater streams), and synthetic graphite production (reference product + graphite fines + crucibles). Without the full enum, the BW export layer cannot correctly handle allocation or substitution for these.

11.2 Q12 — Sign Convention and amount_is_ecoinvent_signed

Decision: Remove FIX-01 (sign-flip rule) entirely. Store ecoinvent amounts as-is. Add `amount_is_ecoinvent_s BOOLEAN DEFAULT FALSE` to `process_exchange`. Enforce two separate validation rules instead of one blanket NEGATIVE_QTY rule.

Rationale: 50.3% of ecoinvent datasets contain negative exchange amounts. Three intentional patterns were confirmed from the corpus scan:

1. **Waste treatment inputs** (43% of negative cases): e.g. `waste polyethylene: −1.0 kg` — negative because waste leaves the economy into treatment.
2. **Avoided burden outputs** (30%): e.g. electricity generated as a co-product of oil production has a negative avoided burden value.
3. **Soil metal uptake** (20%): agricultural processes show negative elementary flows for metals leaving soil into the crop.

Flipping any of these corrupts the mass balance and produces incorrect LCA results.

Two-convention model:

Data source	Sign convention	amount_is_ecoinvent_signed
Manufacturer foreground (manual or SI2 import)	Always positive; direction via `exchange_direction` field. ISO 14040/14044 convention.	FALSE (default)
Ecoinvent `.spold` import	Preserves original sign from `amount` attribute. Negative values are intentional.	TRUE

Sign logic at export time: The BW export layer — not the storage layer — applies the Leontief A-matrix sign convention required by Brightway2 (technosphere inputs negative, reference product output positive). It reads `amount_is_ecoinvent_signed` to know which convention the stored value uses before applying the transformation. See Section 12 for the full export layer specification note.

11.3 Q13 — ActivityNames XML Parser Fix (Pending)

Status: The `scan_ecoinvent_xml.py` scanner incorrectly extracted whitespace from `ActivityNames.xml` because name text is stored in a child element with a language attribute, not as direct element text. This will be fixed in the scanner before BW mapping index construction begins. No schema change is required; this is a parser script issue only.

Impact: ActivityNames is the vocabulary source for TF-IDF fuzzy matching in the BW mapping step. Without correct extraction, the mapping index cannot be built. The fix is straightforward once a sample XML entry is available.

11.4 Q14 — Formula Handling: Use Pre-Computed Amount

Decision: Always use the pre-computed `amount` attribute from the spold XML as `quantity_user`. Store `mathematicalRelation` as `formula_user` for audit and display purposes only. Never re-evaluate formulas at import time.

Rationale: The ecoinvent corpus contains 4,707 unique formula variable names (1,783 global parameters + approximately 2,900 process-local variables). Re-evaluating `mathematicalRelation` would require loading the full parameter tree, resolving variable scoping correctly across global and process-local namespaces, and handling all formula types (arithmetic, variable references, co-product allocation sums, production volume ratios). Ecoinvent has already done this work; the `amount` attribute is the correct, pre-computed result.

The `formula_user` string is retained for two reasons: it explains *why* an exchange has the value it does (useful for auditing), and it enables a future parameter sensitivity feature where a user could modify a global parameter and see which exchanges are affected. However, it is displayed as read-only information only — the stored `quantity_user` is always authoritative.

11.5 Corpus-Driven Seed Data Changes

Two catalog tables are seeded differently from the pre-build baseline:

1. **unit_catalog:** Seeded with 21 exchange units confirmed from the full corpus scan, not the 9 representative units in the pre-build baseline. All 21 units are documented in Section 1.3.
2. **region_catalog:** Seeded with all 541 geographies from `Geographies.xml`, not only the 335 that appear in the current corpus. The extra 206 geographies prevent foreign key failures for rare locations in future ecoinvent releases or manufacturer-defined processes.

12 Brightway Export Layer — Implementation Note

MUST BE RESOLVED BEFORE FULL IMPLEMENTATION

The Brightway export layer is the most technically critical and currently least-specified component of the system. The schema and storage decisions documented in this version are correct, but the precise logic that transforms stored rows into valid Brightway `Activity` and `Exchange` objects has not yet been formally specified.
This section must be completed and reviewed before writing any export code. Implementing the export layer against an incomplete specification is the most likely source of silent calculation errors — results that appear numerically plausible but are wrong because sign conventions, matrix assignment, or waste flow handling were applied incorrectly.

12.1 What needs to be specified

The following questions must be answered in full before implementation begins. They are recorded here so they are not lost and so the person writing the export code has a clear checklist.

1. A-matrix sign convention. Brightway2 constructs a technosphere matrix **A** where, by convention:

- The reference product output of a process appears on the *diagonal* as a **positive** value (production).
- All technosphere *inputs* appear as **negative** values (consumption).
- Co-products and waste outputs are handled differently depending on the system model.

The export layer must apply this convention correctly regardless of what sign is stored in `quantity_user`. The `amount_is_ecoinvent_signed` flag and `exchange_direction` field together determine the transformation. The exact mapping rules must be written out as a lookup table before coding begins.

2. Waste flow matrix assignment (B-matrix vs. A-matrix). In Brightway, elementary flows (biosphere exchanges) go into the **B** matrix; technosphere exchanges go into **A**. However, waste treatment flows (`output_type = waste_output`) are technosphere flows that can appear in either matrix depending on how the system model handles them. The export layer must have an explicit rule for each combination of (`output_type`, `exchange_direction`, `is_elementary_flow`) and which Brightway exchange object it produces.

3. Co-product handling in the cutoff system model. In the cutoff model, co-products (`output_type = coproduct`) are economically allocated or cut off. The export layer must decide: are co-product exchanges written to the Brightway activity at all, and if so, with what sign and amount? Brightway's `bw2io` ecospold2 importer handles this internally — the export layer should either replicate that logic or delegate to it explicitly.

4. Foreground-to-foreground linking. When two foreground processes are linked (one's output is another's input), the export layer must write the correct Brightway exchange objects to resolve the link. The `process_link` table stores these explicit edges. The exact Brightway exchange format for foreground-to-foreground links (vs. foreground-to-background links resolved via `bw_mapping_selection`) must be specified.

5. Uncertainty and distribution parameters. Brightway exchanges carry uncertainty fields (`loc`, `scale`, `minimum`, `maximum`, `uncertainty_type`). The schema stores distribution parameters on `model_parameter` rows but not on individual exchanges. It must be decided: (a) do individual exchanges carry uncertainty in the BW export, and if so, from where? (b) how are the `model_parameter` distributions propagated to exchange-level BW uncertainty fields?

6. Unmappable flows. The current spec says unmappable flows (`mapping_status = unmappable`) are skipped and logged. This must be validated against Brightway's behaviour: does omitting a flow silently affect the calculation or does Brightway raise an error? The export log must clearly identify every skipped exchange so the LCA practitioner can manually add characterisation factors in Brightway post-import.

7. Database name and key format. Brightway activities are identified by `(database, code)` pairs. The export layer must decide the database name used for foreground activities and the format of the `code` field (UUID? process name slug?). This must be consistent with how `bw_mapping_selection` stores `bw_activity_key` for background datasets.

12.2 Recommended approach

Before writing any export code:

1. Write a standalone Python script that imports a known ecoinvent activity into Brightway using `bw2io` and inspects the resulting `Activity` and `Exchange` objects. Record exactly which fields are set and with what values. This is the ground truth for how the export must behave for background datasets.
2. Write the same inspection for a simple foreground process (one output, two inputs, one elementary emission). This covers the foreground case.

3. Write the field mapping table: for each combination of schema fields, what Brightway exchange object is produced. Cover all `output_type` values, both `amount_is_ecoinvent_signed` values, and both `is_elementary_flow` values.
4. Have the mapping table reviewed by someone familiar with the Brightway internals before writing any production export code.

13 Key Design Clarifications

13.1 Admin vs. Manufacturer Responsibilities

Action	Who performs it	How enforced
Create a project	**Admin only**	RLS INSERT policy on `project` checks `role = admin`
Assign users to a project	**Admin only**	`project_member` INSERT restricted to admin; `assigned_by` column NOT NULL
Create a battery model	Manufacturer (within assigned project)	RLS checks `project_member` membership
Create/edit revisions, processes, exchanges	Manufacturer	RLS traces FK chain back through `project_member`
Trigger mapping candidate generation	Manufacturer (auto on import) or manual	`mapping_job` INSERT via `create-flow` completion webhook; Foulan can also re-trigger manually
Freeze a revision	Manufacturer	Sets `frozen_at`; subsequent edits blocked by app layer
Unfreeze a revision	**Admin only**	`unfreeze-revision` Edge Function; appends to `unfreeze_log`; service-role key
Create catalog sets, templates, rules	**Admin only**	RLS INSERT policy checks `role = admin`

13.2 Why artifact and import_job / export_job are separate tables

A common point of confusion is why the file (`artifact`) and the operation (`import_job` / `export_job`) are in different tables.

	artifact	import_job / export_job
What it is	The file.	The operation.
Lifecycle	Permanent. Never deleted (audit trail).	Updated as the job runs.
On failure	Exists if file was uploaded.	Exists; `status=failed`; `artifact_id=NULL`.

Relationships	One artifact can be referenced by multiple jobs (re-imports).	Points to one artifact (the produced file).

13.3 Why bw_mapping_candidate and bw_mapping_selection are separate

	bw_mapping_candidate	bw_mapping_selection
Written by	FastAPI automated matcher.	Foulan (human decision).
Cardinality	Many per flow (ranked list).	Exactly one per flow per revision.
Immutability	Never deleted; audit of what options existed.	Updated if Foulan changes his mind.
Export use	Not used directly.	Export engine reads this only.

13.4 What mapping_status values mean at export time

mapping_status	Export engine action	Example
`mapped`	Write `bw_activity_key` from `bw_mapping_candidate` as background dataset reference.	cobalt sulfate → ecoinvent GLO activity
`foreground`	Resolve to internal `process_instance` by name match. No ecoinvent lookup.	electricity → internal `pi-027`
`unmappable`	Skip exchange; log warning in `export_job.error_log`.	Aerophine → excluded
`pending`	**Abort export.** Raise error listing pending flows.	lithium_hydroxide → blocks export

13.5 How validation_issue links to specific data

Pointer system

`validation_issue` has two nullable FK columns: `process_id` and `exchange_id`. The validation engine populates these to link each issue to the exact entity it concerns:

- **process_id only**: issue is at the process level (e.g. `MISSING_REF_FLOW` — no exchange with `output_type = reference` exists for this process).
- **exchange_id only**: issue is at the exchange level (e.g. `NEGATIVE_FOREGROUND_QTY` — a specific foreground exchange has an unexpected negative value).
- **Both NULL**: issue is at the revision level (e.g. `PARAM_STOCHASTIC` — informational about the whole revision).

The frontend uses these IDs to scroll the AG Grid to the offending row and highlight it.

14 Technology Stack

Overview

The Battery LCA Tool is built on three tiers: a React frontend served statically via Vercel, a Supabase backend handling all persistence and authentication, and a small FastAPI service on Railway for the Python-native processing work (import parsing, export generation, Monte Carlo sampling). Total production infrastructure cost is approximately $32/month. Development is entirely free.

14.1 System Architecture

Data flows left to right through the stack. The browser never communicates directly with the FastAPI service; all requests are routed through Supabase Auth so that Row Level Security is always enforced before any data is touched.

Browser	⟶	**Supabase**	⟶	**FastAPI**	⟶	**Storage**
React + TypeScript		Auth + PostgreSQL		Railway (Python)		Supabase Storage
Vite + shadcn/ui		RLS + Edge Fn		Import / Export / BW		xlsx, json-ld
AG Grid		Storage		Monte Carlo		artifacts

14.2 Frontend Technologies

Technology	Layer	Rationale
React 18	UI framework	The component model maps naturally to the schema hierarchy: Project → Model → Revision → Process → Exchange. Concurrent rendering prevents UI freezing during large table re-renders.
TypeScript	Language	The v2 schema has 18+ tables and 200+ columns. `supabase gen types` auto-generates typed clients directly from the database schema, meaning a column rename in Postgres immediately surfaces as a compile error in the frontend.
Vite	Build tooling	Dramatically faster cold starts and hot module replacement than Create React App. Native ES module serving avoids bundling overhead during development.
shadcn/ui	Component library	Ships components as owned source files rather than a black-box npm package. Essential for the custom UX of exchange tables, validation badges, and formula editors where opinionated libraries like MUI are very hard to override.
Tailwind CSS	Styling	Utility-first; no context-switching between CSS files and components. Purges unused styles at build time so bundle size stays minimal.

AG Grid Community	Exchange table	Free. Handles the 984-row `process_exchange` table with virtual scrolling, inline cell editing, column filtering, and copy-paste from Excel. Spreadsheet-like UX reduces training burden for manufacturers who work in Excel daily.
TanStack Query	Server state	Wraps Supabase JS client calls with caching, background refetching, and optimistic updates. Makes the save-and-revalidate workflow trivial: invalidate the `validation_run` query and it refetches automatically.
Zustand	Client state	Lightweight global state for non-server data: active revision, selected process in sidebar, panel open/closed state. Redux is overkill for this scope.
Zod	Validation	Defines the same constraints as the database as TypeScript schemas that run in the browser, catching errors before a round-trip to the server. Zod schemas mirror Pydantic models in FastAPI, keeping validation consistent across the stack.
React Hook Form	Form management	Integrates with Zod for schema-validated forms. Minimal re-renders keep exchange entry forms fast even with 50+ rows. Handles field arrays (repeating exchange rows) cleanly.
Recharts	Visualisation	Lightweight, React-native charting for parameter sensitivity plots and contribution analysis. Sufficient for bar charts, tornado diagrams, and scatter plots.

14.3 Backend — Supabase

Supabase provides PostgreSQL, authentication, file storage, and Edge Functions as a single managed service. The entire persistence and auth layer is production-ready on day one, allowing the team to focus on LCA domain logic.

Feature	Layer	Rationale
PostgreSQL 15	Database	The v2 schema was designed for PostgreSQL from the outset: UUID PKs, TIMESTAMPTZ, JSONB columns, ENUM types, and partial unique indexes. All native PostgreSQL features unavailable or awkward in MySQL or SQLite.
Row Level Security	Security	RLS policies enforce multi-tenancy at the database level. Even if application code has a bug, a manufacturer can never read another manufacturer's data. Policies reference `auth.uid()` directly, so access control is never bypassed.

Supabase Auth	Authentication	Handles JWT issuance, refresh token rotation, and email delivery for the invite-only manufacturer login model. Admin calls `supabase.auth.admin.inviteUserByEmail()` server-side; the service-role key never reaches the browser.
Supabase Storage	File storage	Stores all `artifact` files (imported xlsx, exported Brightway xlsx, JSON-LD). Storage policies mirror RLS policies using the same JWT, so manufacturers can only download their own project files.
Edge Functions (Deno)	Serverless	Six specific functions: `invite-user` (admin-triggered invite), `on-user-created` (inserts `app_user` row), `create-project` (admin-only project and member creation), `create-flow` (service-role catalog writes for auto-created flows), `trigger-validation` (fires automatically on save with 3-second debounce), `unfreeze-revision` (admin-only; appends to `unfreeze_log`).
Supabase CLI	Local dev	Runs a full local Supabase stack in Docker. Developers work against a local instance; migrations are applied locally first, then pushed to production. Essential for testing RLS policies without touching live data.

14.4 Processing Service — FastAPI on Railway

Supabase is not designed for heavy file processing, formula evaluation, or generating multi-sheet Excel workbooks. These operations are Python-native: Brightway2, openpyxl, and scipy are the standard LCA libraries. A dedicated Python service is the correct architectural choice.

Why not Supabase Edge Functions for processing?

Edge Functions run Deno (TypeScript/JavaScript). There is no Brightway, no openpyxl, no scipy for Monte Carlo sampling. Reimplementing these in JavaScript would be months of work with no benefit. Python is the right tool for LCA computation.

Technology	Layer	Rationale
FastAPI	API framework	Python's highest-performance async web framework. Auto-generates OpenAPI docs from type annotations. Pydantic models mirror Zod schemas on the frontend, keeping validation consistent across the stack.
Python 3.11	Language	Required by Brightway2 and the LCA Python ecosystem. 3.11 offers significant performance improvements for the loop-heavy numerical work in Monte Carlo sampling.

openpyxl	Excel I/O	Reads SI2-format xlsx imports (parsing activity headers, exchange rows, formula strings) and writes the Brightway Excel export. The only Python library that exposes raw formula strings rather than evaluated values, which is required for `formula_user` capture.
Brightway2 / bw2io	LCA export	The target export format. Using the actual Brightway library to validate output before returning it to the client catches mapping or typing errors before the manufacturer downloads the file.
scipy + numpy	Stochastic sampling	Monte Carlo sampling for model parameters with triangular and linear distributions. `scipy.stats.triang` and `numpy.random` are the standard tools for the distributions defined in `model_parameter`.
SQLAlchemy (asyncpg)	DB access	Connects directly to Supabase PostgreSQL with the service-role credentials. Async ORM handles complex queries (loading a full revision with all exchanges for export) more ergonomically than raw SQL strings.
Pydantic v2	Data models	FastAPI's built-in validation layer. Models like `ProcessExchangeImport` implement the schema fix rules as validators (e.g. a validator that flips negative quantities). Pydantic v2 is 5–10× faster than v1 due to Rust-compiled core.
pytest + hypothesis	Testing	pytest covers fixed test cases for all 10 schema fix scenarios. hypothesis generates random xlsx-like structures to catch parser edge cases that fixed tests miss. CI runs these on every push.
Railway	Hosting	Deploys from a GitHub repo with zero server configuration. A single Hobby instance ($5–7/month) handles the load for a small manufacturer user base. Automatic deploys on push; no Dockerfile required.

14.5 Key Architectural Decisions

Decision	Chosen	Alternatives	Reason
Backend language for processing	**Python (FastAPI)**	Node.js, Go	Brightway, openpyxl, scipy are Python-native. Reimplementing in JS or Go would be months of work with no benefit.

Processing host	**Railway**	AWS Lambda, Render, Fly.io, Heroku	Simplest PaaS for Python. Cheaper than Heroku. Lambda has cold-start issues for long xlsx processing.
Frontend state management	**TanStack Query + Zustand**	Redux, MobX, SWR	TanStack Query handles server state; Zustand handles UI state. Redux is overkill; SWR lacks mutation/invalidation ergonomics.
Component library	**shadcn/ui**	MUI, Ant Design, Chakra	Owned source code, not a dependency. Essential for custom exchange table UX. MUI overriding is notoriously painful.
Exchange table widget	**AG Grid Community**	React Table, Handsontable	Free; handles virtualisation and inline editing out of the box. Handsontable has a restrictive commercial licence.
Database	**Supabase / PostgreSQL**	Firebase, PlanetScale, Neon	Schema requires ENUMs, JSONB, partial unique indexes — all PostgreSQL-specific. Firebase cannot model the relational structure.
Auth model	**Supabase Auth (invite-only)**	Custom JWT, Clerk, Auth0	Integrates directly with RLS via `auth.uid()`. Clerk and Auth0 require mapping external user IDs into the DB separately.
File storage	**Supabase Storage**	AWS S3, Cloudflare R2	Uses the same auth JWT as the database. Storage policies mirror RLS policies with no extra configuration.
Project creation	**Admin-only via Edge Function**	Self-service by manufacturers	Ensures consistent project scoping, controlled access assignment, and prevents manufacturers from creating unauthorised workspaces.

14.6 Infrastructure Cost

Service	Free tier	Paid tier	Notes
Supabase	Free (50k MAU, 500MB DB)	Pro: $25/month	Free sufficient for dev. Pro needed above 500MB DB or 50+ active users.
Railway (FastAPI)	Free trial (500 hrs/month)	Hobby: $5–7/month	One always-on Hobby instance covers production load.

Vercel (Frontend)	Free (hobby)	Pro: $20/month	React app is static assets after build. Free tier handles this globally via CDN.
Resend (email)	Free (3,000 emails per month)	15–20 euros per month	Custom SMTP for Supabase Auth invite emails. 3,000 per month far exceeds invite-only volume.
Total production cost: Supabase Pro ($25) + Railway ($7) + Vercel Free ≈ $32/month. Devel			

14.7 Recommended Build Order

Build in dependency order. The import parser is the highest-risk component and is built first so that problems are discovered before the frontend exists and creates pressure to ship.

Phase	Area	Description
1	Schema + infrastructure	Apply the v2 schema migration to a local Supabase instance. Confirm all tables, ENUMs, RLS policies, and constraints create cleanly. Set up GitHub repo, Railway project, Vercel project.
2	Import parser (FastAPI)	Build the openpyxl-based xlsx parser. Write pytest tests for each fix scenario. This is the core domain logic and should be tested exhaustively before any UI exists.
3	Auth + invite flow	Implement the `invite-user` Edge Function and `on-user-created` hook. Test the full invite → login → RLS flow end to end with a test manufacturer account.
3b	Admin project management	Implement the `create-project` Edge Function and the admin UI for creating projects and assigning manufacturers. Test that manufacturers cannot create projects directly.
4	Frontend scaffold	Vite + React + TypeScript + shadcn/ui setup. Generate Supabase types. Build the project/model/revision navigation shell with TanStack Query fetching real data.
5	Exchange entry UI	AG Grid-based `process_exchange` editor. The most complex frontend component: inline editing, validation state highlighting, formula display.
6	BW mapping UI	Flow catalog matching interface. Show mapping candidates, allow selection, set `mapping_status`. Calls the FastAPI `/candidates` endpoint which fuzzy-matches against the ecoinvent index.
7	Export	Wire the FastAPI export endpoint to the UI. Download flow, artifact creation, `export_job` status polling.
8	Parametric / Monte Carlo	Model parameter editor, stochastic sampling endpoint in FastAPI, result visualisation with Recharts.

14.8 Development Tooling

Tool	Purpose	Notes
Supabase CLI	Local dev	Runs full local Supabase stack in Docker. Migrations applied locally first, then pushed. Essential for testing RLS policies without touching live data.
Docker Desktop	Local infra	Required for the Supabase CLI local stack. Also used to run FastAPI locally in an environment matching Railway's container.
ESLint + Prettier	Code quality	TypeScript and React hooks plugins catch missing `useEffect` deps, unsafe `any` usage. Prettier runs on save.
Vitest	Frontend testing	Vite-native test runner, faster than Jest. Unit tests for Zod schemas, utility functions, component logic. React Testing Library for component tests.
GitHub Actions	CI/CD	On every push: type-check, ESLint, Vitest; FastAPI pytest suite. Deploy to Vercel and Railway on merge to main. Supabase migrations applied via CLI in a separate step.
Bruno	API testing	Recommended over Postman: stores collections as local files in the repo rather than in the cloud. Used for manually testing FastAPI endpoints.

15 Admin Panel — Screens, Actions, and Data Access

Overview

The Admin Panel is a dedicated section of the frontend accessible only to users with `role = admin` in `app_user`. It is the control centre for everything a manufacturer cannot do: creating and managing projects, editing the shared configuration tables (catalog, templates, validation rules), managing user accounts, and inspecting or exporting data across all projects. This section documents each screen in the Admin Panel, the database operations it performs, and the RLS or Edge Function boundary that enforces admin-only access.

How admin access is enforced

Every admin-only operation goes through one of two mechanisms:

- **RLS INSERT/UPDATE/DELETE policy:** tables like `project`, `flow_catalog`, `process_template` have policies that check `(SELECT role FROM app_user WHERE user_id = auth.uid()) = 'admin'`. These fire at the database level; even a direct API call from a manufacturer's JWT is rejected.
- **Edge Function with service-role key:** operations that require bypassing RLS entirely (e.g. reading data across all projects for a global export) use a Supabase Edge Function that runs with the service-role key. The function itself checks the caller's JWT for admin role before proceeding.

The frontend enforces admin-only routing (hiding the Admin Panel nav item for non-admins), but this is defence-in-depth only — the backend enforces the real boundary.

15.1 Screen 1 — User Management

Purpose

The User Management screen is where the admin invites new manufacturer users, views all existing accounts, changes roles, and deactivates accounts. It is the entry point for onboarding any new participant to the tool.

15.1.1 User list view

The screen opens with a table of all rows in `app_user`. The admin can see all users because the RLS SELECT policy on `app_user` allows `role = admin` to read all rows (manufacturers can only read their own row).

```sql
SELECT user_id, email, display_name, role, created_at, last_login_at
FROM app_user
ORDER BY created_at DESC;
-- RLS SELECT policy permits this for admin role only.
-- Manufacturers see: WHERE user_id = auth.uid() only.
```

user_id	email	role	created_at	last_login_at
admin-user-uuid	admin@tool.eu	admin	2024-01-01	2024-02-10

usr-Foulan-uuid	Foulan.Foulani@examplecode.de	editor	2024-01-10	2024-02-09
usr-alice-uuid	alice@battery.com	editor	2024-01-15	2024-02-08

15.1.2 Invite a new user

The admin enters an email address and selects a role. The frontend calls the `invite-user` Edge Function.

```
-- Called server-side with service-role key after Supabase Auth invite is
   accepted.
INSERT INTO app_user (user_id, email, display_name, role, created_at)
VALUES (
  auth.uid(),                -- set by Supabase Auth JWT
  'new.manufacturer@co.de',
  'New Manufacturer',
  'editor',                  -- admin chose this role at invite time
  NOW()
);
```

15.1.3 Change a user's role

The admin selects a user and changes their role from the dropdown. This calls the `update-user-role` Edge Function (service-role key required to update another user's row).

```
UPDATE app_user
SET role = 'reviewer'       -- admin selected new role
WHERE user_id = 'usr-alice-uuid';
-- Only accessible via Edge Function with service-role key.
-- A manufacturer cannot UPDATE any app_user row via RLS.
```

15.1.4 Deactivate a user

The tool uses a soft-delete pattern. An `is_active` column on `app_user` is set to FALSE. The RLS policies on all downstream tables check `is_active = TRUE` via a join, so the deactivated user's JWT is silently denied access everywhere without deleting any data.

```
UPDATE app_user
SET is_active = FALSE
WHERE user_id = 'usr-alice-uuid';
-- Downstream RLS policies enforce:
-- AND (SELECT is_active FROM app_user WHERE user_id = auth.uid()) = TRUE
-- so alice's existing JWT immediately loses all access.
```

Deactivation does not delete data

Setting `is_active = FALSE` leaves all of Alice's `battery_model`, `process_instance`, and other rows intact. The admin can still see Alice's data via the Global Data View (Screen 5). Re-activating the account restores full access.

15.2 Screen 2 — Project Management

Purpose

The Project Management screen is where the admin creates new projects, assigns manufacturers to them, adjusts membership, and archives completed projects. Manufacturers never see this screen.

15.2.1 Project list view

The admin sees all projects. The RLS SELECT policy on `project` allows admins to see all rows unconditionally; manufacturers only see rows where they have a `project_member` entry.

```sql
SELECT
  p.project_id,
  p.name,
  p.created_at,
  p.archived,
  COUNT(pm.user_id) AS member_count,
  COUNT(bm.model_id) AS model_count
FROM project p
LEFT JOIN project_member pm ON pm.project_id = p.project_id
LEFT JOIN battery_model bm ON bm.project_id = p.project_id
GROUP BY p.project_id
ORDER BY p.created_at DESC;
```

15.2.2 Create a new project

The admin fills in a project name and description, then selects one or more manufacturer users to assign immediately. The `create-project` Edge Function handles both the `project` INSERT and the initial `project_member` INSERT(s) atomically.

Edge Function: create-project (atomic project + member creation)

```sql
-- Step 1: insert the project record
INSERT INTO project
  (project_id, name, description, created_by, created_at, archived)
VALUES (
  gen_random_uuid(),
  'LFP Cell Assembly Study 2025',
  'LCA study for prismatic LFP cell assembly line',
  'admin-user-uuid',     -- the calling admin's auth.uid()
  NOW(),
  FALSE
)
RETURNING project_id INTO v_project_id;

-- Step 2: assign selected manufacturers
INSERT INTO project_member
  (project_id, user_id, role, assigned_by, created_at)
VALUES
  (v_project_id, 'usr-alice-uuid', 'manufacturer', 'admin-user-uuid',
      NOW()),
  (v_project_id, 'usr-Foulan-uuid', 'manufacturer', 'admin-user-uuid',
      NOW());
```

```
-- Both steps are wrapped in a transaction; either both succeed or neither
   does.
```

15.2.3 Manage project membership

From the project detail view, the admin can add or remove manufacturers. Adding creates a new `project_member` row; removing deletes it (the only table in the schema where DELETE is permitted — membership rows carry no audit-critical data of their own).

```
INSERT INTO project_member
  (project_id, user_id, role, assigned_by, created_at)
VALUES (
  'proj-0001-uuid',
  'usr-alice-uuid',
  'reviewer',            -- Alice gets read-only access to this project
  'admin-user-uuid',
  NOW()
);
```

```
DELETE FROM project_member
WHERE project_id = 'proj-0001-uuid'
  AND user_id    = 'usr-alice-uuid';
-- Alice immediately loses RLS access to all data in proj-0001.
-- Her data rows (battery_model, process_instance etc.) are NOT deleted.
```

15.2.4 Archive a project

Archiving sets `archived = TRUE`. The application layer treats archived projects as read-only: all INSERT/UPDATE/DELETE on `battery_model` and downstream tables for that project are blocked. The project and all its data remain queryable.

```
UPDATE project SET archived = TRUE
WHERE project_id = 'proj-0001-uuid';
-- App layer check added to all downstream RLS write policies:
-- AND (SELECT archived FROM project WHERE project_id = ...) = FALSE
```

15.3 Screen 3 — Catalog Management

Purpose

The Catalog Management screen gives the admin full CRUD access to the shared configuration tables: `flow_catalog`, `flow_allowed_unit`, `unit_catalog`, `region_catalog`, `data_origin_catalog`, `catalog_set`, `process_template`, and `template_expected_exchange`. Manufacturers have SELECT-only access to all of these tables via RLS; only admins can INSERT, UPDATE, or DELETE rows here.

Changes here affect all projects immediately

Because these tables are shared across all projects (under the DEFAULT catalog), any change the admin makes is immediately visible to all logged-in manufacturers. Adding a new flow to `flow_catalog` will appear in every manufacturer's flow dropdown on next page load. Renaming a flow or changing a unit's conversion factor will silently affect any revision that references it. The admin should treat these tables with the same care as a shared database migration.

15.3.1 Flow Catalog tab

The admin sees all rows in `flow_catalog` in an editable AG Grid. Columns shown: `canonical_name`, `display_name`, `kind`, `dimension`, `default_unit`, `is_elementary_flow`, `cas_number`. Actions available:

Edit an existing flow.

```
UPDATE flow_catalog
SET
  display_name = 'Nickel Sulfate Hexahydrate (primary)',
  cas_number   = '10101-97-0'   -- corrected CAS
WHERE flow_id = 'flw-niso4-uuid';
```

Merge near-duplicate flows. When two manufacturers have independently created flows that represent the same substance (e.g. `NiSO4` and `nickel_sulfate_primary`), the admin can merge them. This is a two-step operation: re-point all FK references to the canonical flow, then deprecate the duplicate.

```
-- Step 1: re-point all references from the duplicate to the canonical flow
UPDATE process_exchange
  SET flow_id = 'flw-niso4-uuid'         -- canonical
  WHERE flow_id = 'flw-niso4-dup-uuid'; -- duplicate

UPDATE bw_mapping_selection
  SET flow_id = 'flw-niso4-uuid'
  WHERE flow_id = 'flw-niso4-dup-uuid';

UPDATE bw_mapping_candidate
  SET flow_id = 'flw-niso4-uuid'
  WHERE flow_id = 'flw-niso4-dup-uuid';

-- Step 2: soft-delete the duplicate (set deprecated = TRUE rather than
    DELETE)
UPDATE flow_catalog
  SET deprecated = TRUE
  WHERE flow_id = 'flw-niso4-dup-uuid';
-- The deprecated flag is checked by the UI dropdown query:
-- WHERE deprecated IS NOT TRUE
```

Add a new flow manually. The admin can pre-create flows rather than waiting for a manufacturer import to auto-create them. Useful when a known substance should appear in the dropdown before any manufacturer has entered it.

```sql
INSERT INTO flow_catalog
  (flow_id, catalog_set_id, canonical_name, display_name,
   kind, dimension, default_unit, is_elementary_flow, cas_number)
VALUES (
  gen_random_uuid(),
  'cat-0001-uuid',
  'lithium_carbonate_primary',
  'Lithium Carbonate (primary)',
  'material', 'mass', 'kg', FALSE, '554-13-2'
);

-- Immediately create the allowed units for this flow:
INSERT INTO flow_allowed_unit (flow_id, unit_id)
SELECT 'new-flow-uuid', unit_id
FROM unit_catalog
WHERE dimension = 'mass';
```

15.3.2 Unit Catalog tab

The admin sees all rows in `unit_catalog` in a read-mostly grid. Editing is restricted to `display_name` and `description` only — changing `factor_to_si` for an existing unit would silently corrupt all stored `quantity_user` values that used that unit. The UI disables the `factor_to_si` cell for existing rows. New rows can be added freely.

```sql
INSERT INTO unit_catalog
  (unit_id, symbol, dimension, factor_to_si, description)
VALUES (
  gen_random_uuid(),
  't',           -- metric tonne
  'mass',
  1000.0,        -- 1 tonne = 1000 kg
  'Metric tonne'
);
```

15.3.3 Region Catalog tab

The admin can add non-standard geographic codes that are not in ISO 3166-1 but are needed for specific ecoinvent locations (e.g. `IAI Area, EU27 & EFTA`).

```sql
INSERT INTO region_catalog (code, name, is_ecoinvent_shortcut)
VALUES ('IAI-EU27', 'IAI Area, EU27 and EFTA', TRUE);
```

15.3.4 Data Origin Catalog tab

The admin can add new provenance categories. This is the simplest catalog table — a free text code and label only.

```sql
INSERT INTO data_origin_catalog (code, label, description)
VALUES (
  'simulation',
  'Simulation',
  'Value derived from a process simulation model (e.g. Aspen, ProSim)'
);
```

15.3.5 Validation Rules tab

The admin can view all active validation rules, edit their `severity` (e.g. promote a `warning` to an `error`), edit the `rule_json` blob that governs what the engine checks, and add entirely new rules. Changes take effect on the next validation run for any revision; the `config_hash` in `validation_run` will change accordingly.

```sql
-- The team has decided that unmapped flows should block export entirely.
UPDATE validation_rule
SET severity = 'error'
WHERE code = 'UNMAPPED_FLOW';
-- All future validation runs will now fail if any flow is unmapped.
-- Existing validation_run rows are unaffected (they record the old
    config_hash).
```

```sql
INSERT INTO validation_rule (code, severity, description, rule_json)
VALUES (
  'MISSING_DATA_ORIGIN',
  'warning',
  'Check that every process_exchange has a non-null data_origin_id',
  '{"check": "process_exchange.data_origin_id IS NOT NULL",
    "scope": "exchange",
    "message": "Exchange {exchange_id} has no data origin recorded."}'
);
```

15.3.6 Process Templates tab

The admin can view, create, and edit process templates and their expected exchange slots. This is the primary mechanism for ensuring that manufacturers entering common process types get consistent, pre-populated exchange rows.

Create a new template.

INSERT: process_template — admin creates a new template

```sql
INSERT INTO process_template
  (template_id, catalog_set_id, stage, canonical_name, ui_label,
      ui_helptext)
VALUES (
  gen_random_uuid(),
  'cat-0001-uuid',
  'manufacturing',
  'electrolyte_filling',
```

```
  'Electrolyte Filling',
  'Injection of liquid electrolyte into the cell under inert atmosphere. '
  'Requires electrolyte solvent inputs and an inert gas input.'
)
RETURNING template_id INTO v_template_id;
```

Add expected exchange slots to a template.

```
INSERT INTO template_expected_exchange
  (expected_id, template_id, flow_id, direction,
   is_required, group_key, min_occurs, max_occurs, display_order, ui_label)
VALUES
  (gen_random_uuid(), v_template_id,
   (SELECT flow_id FROM flow_catalog WHERE
       canonical_name='electrolyte_solution'),
   'input', TRUE, 'electrolyte', 1, 1, 1, 'Electrolyte solution'),

  (gen_random_uuid(), v_template_id,
   (SELECT flow_id FROM flow_catalog WHERE canonical_name='argon_gas'),
   'input', FALSE, 'inert_gas', 0, NULL, 2, 'Inert gas (if applicable)'),

  (gen_random_uuid(), v_template_id,
   (SELECT flow_id FROM flow_catalog WHERE canonical_name='filled_cell'),
   'output', TRUE, 'reference', 1, 1, 3, 'Filled cell (reference flow)');
```

Edit an existing template slot.

```
UPDATE template_expected_exchange
SET is_required = TRUE, min_occurs = 1
WHERE expected_id = 'tee-005';  -- wastewater slot previously optional
```

15.4 Screen 4 — Global Data View

Purpose

The Global Data View gives the admin a read-only, cross-project view of all data in the system. It is the primary tool for quality oversight, deduplication review, and cross-manufacturer comparison. Manufacturers never see data outside their own assigned projects; the admin always can.

15.4.1 All Projects Summary

A high-level dashboard table showing every project with key statistics.

Query: global project summary

```
SELECT
  p.name                          AS project_name,
  p.created_at,
  p.archived,
  au.display_name                 AS created_by,
```

```sql
  COUNT(DISTINCT pm.user_id)      AS manufacturers_assigned,
  COUNT(DISTINCT bm.model_id)     AS models,
  COUNT(DISTINCT bmr.revision_id) AS revisions,
  COUNT(DISTINCT pi.process_id)   AS processes,
  COUNT(DISTINCT pe.exchange_id)  AS exchanges
FROM project p
JOIN app_user au ON au.user_id = p.created_by
LEFT JOIN project_member pm  ON pm.project_id = p.project_id
LEFT JOIN battery_model bm   ON bm.project_id = p.project_id
LEFT JOIN battery_model_revision bmr ON bmr.model_id = bm.model_id
LEFT JOIN process_instance pi ON pi.revision_id = bmr.revision_id
LEFT JOIN process_exchange pe ON pe.process_id = pi.process_id
GROUP BY p.project_id, au.display_name
ORDER BY p.created_at DESC;
-- Runs via Edge Function with service-role key; bypasses manufacturer RLS.
```

15.4.2 Cross-Project Flow Deduplication Review

The admin can see which flow names appear more than once in `flow_catalog` under the same catalog set, flagging near-duplicates created by different manufacturers during independent imports.

```sql
-- Flows that share a CAS number but have different canonical_names
-- (same substance, registered twice under different names)
SELECT
  f1.canonical_name  AS name_a,
  f2.canonical_name  AS name_b,
  f1.cas_number,
  f1.kind,
  COUNT(DISTINCT pe1.process_id) AS used_in_processes_a,
  COUNT(DISTINCT pe2.process_id) AS used_in_processes_b
FROM flow_catalog f1
JOIN flow_catalog f2
  ON  f1.cas_number = f2.cas_number
  AND f1.flow_id    < f2.flow_id     -- avoid self-join and duplicates
  AND f1.catalog_set_id = f2.catalog_set_id
LEFT JOIN process_exchange pe1 ON pe1.flow_id = f1.flow_id
LEFT JOIN process_exchange pe2 ON pe2.flow_id = f2.flow_id
WHERE f1.cas_number IS NOT NULL
GROUP BY f1.flow_id, f2.flow_id
ORDER BY f1.cas_number;
```

15.4.3 Revision Status Dashboard

The admin can see the validation and export status of every active revision across all projects — useful for tracking which manufacturers are blocked on validation errors or pending mappings.

Query: revision status across all projects

```sql
SELECT
  p.name                  AS project,
  bm.name                 AS model,
  bmr.revision_number,
  bmr.status,
  bmr.frozen_at,
```

```sql
  vr.status                AS last_validation,
  vr.issue_count,
  COUNT(bws.mapping_id) FILTER (WHERE bws.mapping_status = 'pending')
                           AS pending_mappings,
  MAX(ej.exported_at)      AS last_export_at
FROM battery_model_revision bmr
JOIN battery_model bm ON bm.model_id = bmr.model_id
JOIN project p ON p.project_id = bm.project_id
LEFT JOIN LATERAL (
  SELECT status, issue_count
  FROM validation_run
  WHERE revision_id = bmr.revision_id
  ORDER BY run_at DESC LIMIT 1
) vr ON TRUE
LEFT JOIN bw_mapping_selection bws ON bws.revision_id = bmr.revision_id
LEFT JOIN export_job ej ON ej.revision_id = bmr.revision_id
WHERE bmr.is_active = TRUE
GROUP BY p.project_id, bm.model_id, bmr.revision_id, vr.status,
    vr.issue_count
ORDER BY p.name, bm.name;
```

15.4.4 Mapping Coverage Report

The admin can see, across all active revisions, how many flows are mapped vs. pending vs. unmappable — useful for identifying flows that consistently fail automated matching and may need manual catalog enrichment.

```sql
SELECT
  fc.canonical_name,
  fc.kind,
  COUNT(*) FILTER (WHERE bws.mapping_status = 'mapped')      AS mapped_count,
  COUNT(*) FILTER (WHERE bws.mapping_status = 'foreground') AS
      foreground_count,
  COUNT(*) FILTER (WHERE bws.mapping_status = 'pending')     AS
      pending_count,
  COUNT(*) FILTER (WHERE bws.mapping_status = 'unmappable') AS
      unmappable_count
FROM flow_catalog fc
JOIN bw_mapping_selection bws ON bws.flow_id = fc.flow_id
GROUP BY fc.flow_id
HAVING COUNT(*) FILTER (WHERE bws.mapping_status = 'pending') > 0
ORDER BY pending_count DESC;
-- Flows with the most unresolved pending mappings appear first.
-- Admin can use this to prioritise adding BW candidates for common flows.
```

15.5 Screen 5 — Data Export

Purpose

The Data Export screen lets the admin extract data from the system in bulk for purposes outside the tool: regulatory reporting, cross-study aggregation, database backups, or handing off data to an external LCA practitioner. All exports are generated server-side by a FastAPI endpoint and stored as `artifact` rows, so every export is logged and traceable.

15.5.1 Export a single revision (Brightway format)

The admin can trigger a Brightway export for any revision in any project, not just their own. This is equivalent to what Foulan triggers from his UI in Step 7, but available without being a project member.

```
-- insert export_job on behalf of any revision
INSERT INTO export_job
  (export_id, revision_id, format, exported_by, exported_at, status)
VALUES (
  gen_random_uuid(),
  'rev-0001-uuid',     -- any revision, regardless of project membership
  'brightway_xlsx',
  'admin-user-uuid',   -- exported_by = admin
  NOW(),
  'running'
);
-- FastAPI export endpoint accepts admin JWT and bypasses project_member
    check.
```

15.5.2 Export full project data as JSON

For backup or handoff purposes, the admin can export a complete JSON snapshot of all data in a project: all models, revisions, parameters, processes, exchanges, mappings, validation runs, and artifacts metadata. This is generated by a dedicated FastAPI endpoint (`/admin/export/project/{project_id}`).

```
-- The endpoint runs these queries and assembles the result into one JSON
    object:
SELECT * FROM battery_model WHERE project_id = :project_id;
SELECT bmr.* FROM battery_model_revision bmr
  JOIN battery_model bm ON bm.model_id = bmr.model_id
  WHERE bm.project_id = :project_id;
SELECT mp.* FROM model_parameter mp
  JOIN battery_model_revision bmr ON bmr.revision_id = mp.revision_id
  JOIN battery_model bm ON bm.model_id = bmr.model_id
  WHERE bm.project_id = :project_id;
-- ... same join pattern for process_instance, process_exchange,
--     process_link, bw_mapping_selection, validation_run, artifact
-- Result is serialised to JSON and stored as artifact type = 'admin_export'
INSERT INTO artifact
  (artifact_id, revision_id, artifact_type, filename,
   storage_path, mime_type, checksum_sha256, created_at)
VALUES (
  gen_random_uuid(),
  NULL,               -- admin exports are not tied to a single revision
  'admin_export',
  'project_proj-0001_full_export_2025-02-10.json',
  'admin/exports/proj-0001/full_2025-02-10.json',
  'application/json', 'sha256:...', NOW()
);
```

15.5.3 Export catalog tables as CSV

The admin can download any of the shared catalog tables as a CSV file for offline editing, review, or versioning in a spreadsheet. The export covers `flow_catalog`, `unit_catalog`, `region_catalog`, `data_origin_catalog`, and `validation_rule`.

```sql
-- Example: flow_catalog export for external review
SELECT
  fc.flow_id, fc.canonical_name, fc.display_name,
  fc.kind, fc.dimension, fc.default_unit,
  fc.is_elementary_flow, fc.cas_number,
  array_agg(fau.unit_symbol ORDER BY fau.unit_symbol)
    AS allowed_units
FROM flow_catalog fc
LEFT JOIN flow_allowed_unit fau ON fau.flow_id = fc.flow_id
WHERE fc.catalog_set_id = 'cat-0001-uuid'
  AND (fc.deprecated IS NULL OR fc.deprecated = FALSE)
GROUP BY fc.flow_id
ORDER BY fc.kind, fc.canonical_name;
-- Serialised to CSV by FastAPI, returned as a file download.
```

15.5.4 Import catalog updates from CSV

The admin can upload a modified catalog CSV (e.g. after offline review and correction of flow names or CAS numbers) and apply the changes in bulk. The FastAPI `/admin/catalog/import` endpoint validates each row against the schema constraints before applying any changes, reporting errors without partial writes.

```sql
-- For each row in the uploaded CSV:
-- 1. If flow_id exists in flow_catalog: UPDATE non-key fields only
-- 2. If flow_id is new: INSERT with new gen_random_uuid()
-- 3. If a row in the CSV has deprecated = TRUE: set deprecated flag
-- Applied in a single transaction; any constraint violation rolls back all
    changes.

UPDATE flow_catalog
SET
  display_name        = :display_name,
  cas_number          = :cas_number,
  is_elementary_flow  = :is_elementary_flow
WHERE flow_id = :flow_id
  AND catalog_set_id = 'cat-0001-uuid';
```

15.6 Admin Panel — RLS and Edge Function Summary

The table below consolidates which screens require which enforcement mechanism for each database operation.

Screen / Action	Table(s) affected	Mechanism	Policy / Function
View all users	`app_user`	RLS SELECT	Admin sees all; manufacturer sees own row only

Invite user	`app_user` (on accept)	Edge Function	`invite-user` + `on-user-created`; service-role key
Change role / deactivate	`app_user`	Edge Function	`update-user-role` / `deactivate-user`; service-role key
View all projects	`project`	RLS SELECT	Admin: all rows; manufacturer: `project_member` rows only
Create project	`project`	RLS INSERT	`CHECK (role = 'admin')`
Assign member	`project_member`	RLS INSERT	`CHECK (role = 'admin')`
Remove member	`project_member`	RLS DELETE	`CHECK (role = 'admin')`
Archive project	`project`	RLS UPDATE	`CHECK (role = 'admin')`
Edit flow catalog	`flow_catalog`	RLS UPDATE	`CHECK (role = 'admin')`
Add flow	`flow_catalog`	RLS INSERT	`CHECK (role = 'admin')`
Merge/deprecate flow	`flow_catalog` + FKs	Edge Function	`merge-flows`; atomic transaction
Edit unit catalog	`unit_catalog`	RLS UPDATE	`CHECK (role = 'admin')`; `factor_to_si` column locked
Edit region/origin catalogs	`region_catalog`, `data_origin_catalog`	RLS INSERT/UPDATE	`CHECK (role = 'admin')`
Edit validation rules	`validation_rule`	RLS INSERT/UPDATE	`CHECK (role = 'admin')`
Create/edit templates	`process_template`, `template_expected_exchange`	RLS INSERT/UPDATE	`CHECK (role = 'admin')`
Global data view	All tables (cross-project)	Edge Function	Service-role key; admin JWT check inside function
Revision status dashboard	Multiple tables (cross-project)	Edge Function	Service-role key; admin JWT check inside function
Bulk project JSON export	All project tables	Edge Function	`/admin/export/project/{id}`; service-role key
Catalog CSV export/import	Catalog tables	Edge Function	`/admin/catalog/export` and `/admin/catalog/import`

15.7 Build Order Addition — Admin Panel Phases

The Admin Panel phases slot into the overall build order as follows. They can be built in parallel with the manufacturer frontend phases (4–8) since they share the same database and Edge Function infrastructure but are on a separate route.

Phase	Area	Description

A1	Admin scaffold	Add admin role check to frontend routing. Build the Admin Panel shell with sidebar navigation: Users / Projects / Catalog / Data View / Export. Confirm that a manufacturer JWT cannot reach any admin route.
A2	User management	Build the user list, invite form (calls `invite-user` Edge Function), role-change and deactivate actions. Test that deactivating a user immediately revokes their RLS access.
A3	Project management	Build the project list, create-project form (calls `create-project` Edge Function with atomic member insertion), member management (add/remove), and archive toggle.
A4	Catalog management	Build editable AG Grid views for `flow_catalog`, `unit_catalog`, `region_catalog`, `data_origin_catalog`, and `validation_rule`. Include the flow merge/deprecate workflow. Build the process template and expected exchange editor.
A5	Global data view	Build the cross-project dashboards (project summary, revision status, mapping coverage). These call FastAPI endpoints that run with the service-role key.
A6	Data export	Build the export controls: single-revision BW export, full project JSON export, catalog CSV export/import. Wire to the corresponding FastAPI `/admin/export/*` and `/admin/catalog/*` endpoints.